

说明

青稞 V3 系列微处理器是一款基于标准 RISC-V 指令集架构研发的 32 位通用 MCU 微处理器。该系列包括 V3A、V3B、V3C、V3F 和 V3V 五个型号：

- V3A：支持 RV32IMAC 标准指令集扩展。
- V3B：支持 RV32I[M] C[B] 标准指令集，及 XW 自定义指令集扩展，其中部分芯片不支持“M”、“B”指令。
- V3C：支持 RV32IMCB 标准指令集，及 XW 自定义指令集扩展。
- V3F：支持 RV32IMAFCB 标准指令集，及 XW 自定义指令集扩展。
- V3V：支持 RV32IMACB_Zve64x_zvbb 标准指令集，及 XW 自定义指令集扩展。

此外，该系列微处理器还具备多项特色功能，包括自动堆栈保护（HPE）、免表中断（VTF）、精简型单/双线调试接口、以及“WFI”、“WFE”睡眠模式。

特点

特点	描述
指令集架构	RV32I[M][A][F]C[B][V][XW]
流水线	3 级
FPU	V3F 支持标量浮点运算
分支预测	静态分支预测
中断	支持异常在内的共 256 个中断，支持免表中断
自动堆栈保护	最大支持 2 级自动堆栈保护
物理内存保护	支持
低功耗模式	支持睡眠和深度睡眠模式，并支持 WFI 和 WFE 睡眠方式
扩展指令集	支持
调试	单/双线调试接口，标准 RISC-V 调试

第 1 章 概览

青稞 V3 系列微处理器包括 V3A、V3B、V3C、V3F、V3V，各系列之间根据应用场合存在一定的差异，具体的差异详见表 1-1。

表 1-1 青稞 V3 微处理器概览

特点 型号	指令集	硬件 堆栈 级数	中断 嵌套 级数	免表 中断 通道数	流水线	向量表 模式	扩展 指令 (XW)	内存 保护 区域 个数
V3A	RV32IMAC	2	2	4	3	指令	×	×
V3B	RV32I [M] C[B]	2	2	4	3	地址/指令	√	×
V3C	RV32IMCB	2	2	4	3	地址/指令	√	4
V3F	RV32IMAFCB	2	2	4	3	地址/指令	√	4
V3V	RV32IMACBV	2	2	4	3	地址/指令	√	4

注：操作系统的任务切换一般使用软件堆栈，软件堆栈不限级数

1.1 指令集

青稞 V3 系列微处理器遵循标准的 RISC-V 指令集架构，详细的标准文档可参考 RISC-V 基金会官网的指令集手册。RISC-V 指令集架构简洁，支持模块化设计，可以根据不同的需求进行灵活的组合，青稞 V3 系列微处理器支持如下指令集扩展：

- RV32：32 位架构，通用寄存器位宽为 32 位
- I：支持整型操作，具有 32 个整型寄存器
- M：支持整型乘法和除法指令
- A：支持原子指令，其中 V3A 对 lr、sc 指令进行了简单化处理，仅作为 lw、sw 执行，且 sc 结果返回总是成功
- C：支持 16 位压缩指令
- B：支持位操作指令
- F：支持单精度浮点指令
- V：支持 zve64x 和 zvbb 指令集，向量寄存器位宽 64 位
- XW：自扩展字节和半字操作的 16 位压缩指令

注：1：不同型号支持的子指令集可能不同，具体可参考表 1-1；

2：为进一步提高代码密度，扩展 XW 子集，增加以下压缩指令 c.lbu/c.lhu/c.sb/c.sh/c.lbusp/c.lhusp/c.sbsp/c.shsp，使用时需要基于 MRS 编译器或者其提供的工具链；

3：V3B/C 支持内存拷贝指令，具体使用方法可参考库函数且配合 MRS 编译器或其提供的工具链。

1.2 寄存器组

RV32I 拥有 x0-x31 共 32 个寄存器组。在 RV32 中，每个寄存器均为 32 位。下表 1-2 列举了 RV32I 的寄存器及其描述。

表 1-2 RV32I 通用寄存器表

寄存器	ABI 名称	描述	存储者
x0	zero	硬编码 0	-
x1	ra	返回地址	Caller
x2	sp	栈指针	Callee
x3	gp	全局指针	-

x4	tp	线程指针	-
x5-7	t0-2	临时寄存器	Caller
x8	s0/fp	保存寄存器/帧指针	Callee
x9	s1	保存寄存器	Callee
x10-11	a0-1	函数参数/返回值	Caller
x12-17	a2-7	函数参数	Caller
x18-27	s2-11	保存寄存器	Callee
x28-31	t3-6	临时寄存器	Caller

上表中 Caller 属性意为被调过程不保存该寄存器值，Callee 属性意为被调过程保存该寄存器。

V3F 型号内核支持 F 指令集，拥有 F0-F31 共 32 个浮点寄存器，用于存储单精度浮点指令所需数据。下表 1-3 列举了浮点寄存器及其描述。

表 1-3 RISC-V3F 寄存器

寄存器	ABI 名称	描述	存储者
f0-7	ft0-7	浮点临时寄存器	Caller
f8-9	fs0-1	浮点保存寄存器	Callee
f10-11	fa0-1	浮点函数参数/返回值	Caller
f12-17	fa2-7	浮点函数参数	Caller
f18-27	fs2-11	浮点保存寄存器	Callee
f28-31	ft8-11	浮点临时寄存器	Caller

上表中 Caller 属性意为被调过程不保存该寄存器值，Callee 属性意为被调过程保存该寄存器。

V3V 型号内核支持 V 指令集，拥有 V0-V31 共 32 个向量寄存器组，向量寄存器位宽 64bit，支持 64/32/16/8bit 宽度的向量操作。

表 1-4 RISC-V3V 寄存器

寄存器	ABI 名称	描述	存储者
V0-V31	v0-v31	向量寄存器	Callee

注：向量寄存器在被调用过程中支持标准 ABI 和变体 ABI 两种保存形式，详细 ABI 调用可参考《RISC-V ABIs Specification v1.1》及 CSR.vcontrol 寄存器配置。

1.3 特权模式

标准 RISC-V 架构包括三种特权模式：机器模式、监督模式、用户模式，如下表 1-5 所示。其中机器模式为必须实现的模式，其他的模式为可选择实现的模式。详细的可以参考 RISC-V 特权架构标准文档，用户可在 RISC-V 基金会官网免费下载。

表 1-5 RISC-V 架构特权模式

编码	名称	简称
0b00	用户模式	U
0b01	监督模式	S
0b10	保留	保留
0b11	机器模式	M

青稞 V3 系列微处理器支持其中两种特权模式：

- 机器模式

机器模式具有最高的权限，该模式下程序可以访问所有的控制和状态寄存器（Control and Status Register, CSR），同时也能够访问所有的物理地址区域。上电默认处于机器模式下，当执行 mret（机器模式返回指令）返回后，根据 CSR 寄存器 mstatus（机器模式状态寄存器）中 MPP 位，若 MPP=0b00，则退出机器模式进入用户模式，MPP=0b11，则继续保留机器模式。

- 用户模式

用户模式具有最低权限，该模式下只能访问限定的 CSR 寄存器。当发生异常或中断时，微处

理器会由用户模式进入机器模式，处理异常和中断。

1.4 CSR 寄存器

RISC-V 架构中定义了一系列 CSR 寄存器，用于控制和记录微处理器的运行状态。这些 CSR 使用内部专用的 12 位地址编码空间，可扩展 4096 个寄存器。并且使用高两位 CSR[11:10]来定义该寄存器的读写权限，0b00、0b01、0b10 表示允许读写，0b11 表示仅读。使用 CSR[9:8]两位来定义可以访问该寄存器的最低特权级别，该值和表 1-5 中定义的特权模式对应。青稞 V3 微处理器实现的 CSR 寄存器详见第 8 章。

第 2 章 异常

异常（Exception）机制，即拦截和处理“不寻常运行事件”的机制。青稞 V3 系列微处理器搭载一个异常响应系统，能够处理中断在内的多达 256 个异常。当发生异常或中断时，微处理器能够快速响应并处理异常和中断事件。

2.1 异常类型

不管发生异常或是中断，微处理器的硬件行为是一致的，微处理器暂停当前程序，转向异常或中断处理程序，处理完成后返回之前暂停的程序。广义的来说，中断也是异常的一部分。具体当前发生的是中断还是异常可以通过机器模式异常原因寄存器 mcause 来查看。其中 mcause[31] 为 interrupt 域，用于表示产生异常的原因是中断还是异常，mcause[31]=1 表示为中断，mcause[31]=0 表示为异常。mcause[30:0] 为异常编码，用于表示异常产生的具体原因或中断编号，如下表所示。

表 2-1 V3 微处理器异常编码

interrupt	异常编码	同步/异步	异常原因
1	0-1	-	保留
1	2	精确异步	NMI 中断
1	3-11	-	保留
1	12	精确异步	SysTick 中断
1	13	-	保留
1	14	同步	软件中断
1	15	-	保留
1	16-255	精确异步	外部中断 16-255
0	0	同步	指令地址不对齐
0	1	同步	取指令访问错误
0	2	同步	非法指令
0	3	同步	断点
0	4	同步	Load 指令访存地址不对齐
0	5	非精确异步	Load 指令访存错误
0	6	同步	Store/AMO 指令访存地址不对齐
0	7	非精确异步	Store/AMO 指令访存错误
0	8	同步	用户模式下环境调用
0	11	同步	机器模式下环境调用

表中“同步”是指可以准确地定位到某一条执行的指令，例如一条 ebreak 或 ecall 指令，每次执行到该指令均会触发进入异常。“异步”是指不能准确定位于某条指令，每次异常发生时的指令 PC 值有可能不同。其中的“精确异步”是指异常发生后可以精确地定位到某一条指令的边界，保存当前异常发生时的所有内核状态值，当异常处理完成后可以恢复原本的指令继续执行程序，例如外部的中断。而“非精确异步”是指无法精确定位到某一条指令的边界，有可能是某条指令执行了一半被打断后的状态，异常处理完成后无法恢复原程序正常运行，例如访存错误。访问存储器需要时间，微处理器访存时一般不会一直等待访存结束，而是继续执行指令，此时再出现访存错误异常时，微处理器已经执行了后续的指令，无法精确定位或保存完整的异常指令前的寄存器值。

2.2 进入异常

当程序在正常运行过程中，若因某种原因，触发进入异常或者中断。此时微处理器的硬件行为可以概括如下：

（1）暂停当前程序流，转向执行异常或中断处理函数。

异常或中断函数的入口基地址及寻址方式由异常入口基地址寄存器 mtvec 定义，mtvec[31:2] 定义了异常或中断函数的基地址。

mtvec[1:0]定义了处理函数的寻址方式，当 mtvec[0]=0，所有异常和中断使用统一入口，即发生异常或中断时，转向 mtvec[31:2]定义的基地址处执行；当 mtvec[0]=1，异常和中断使用向量表模式，即对每个异常和中断进行编号，根据中断编号*4 进行地址偏移，发生异常或中断时，转向 mtvec[31:2]定义的基地址+中断编号*4 处执行。中断向量表处存放是一条跳转至中断处理函数的指令，也可以是其他指令。

mtvec[1]定义了入口地址中存储的是指令还是地址，当 mtvec[1]=0 是，异常入口地址存储的是跳转指令；当 mtvec[1]=1 时，异常入口地址存储的是跳转地址。

(2) 更新 CSR 寄存器

当进入异常或中断时，微处理器会自动更新相关的 CSR 寄存器，包括机器模式异常原因寄存器 mcause、机器模式异常指针寄存器 mepc、机器模式异常值寄存器 mtval、机器模式状态寄存器 mstatus。

● 更新 mcause

如前所述，进入异常或中断后，其值反映当前的异常种类或中断的编号，软件可以读取该寄存器值查看引起异常的原因或判断中断的来源，详见表 2-1。

● 更新 mepc

标准定义退出异常或中断后微处理器的返回地址保存在 mepc 中。所以当发生异常或中断后，硬件自动更新 mepc 值为当前遇到异常时的指令 PC 值，或中断前下一条预执行的指令 PC 值。异常或中断处理结束后，微处理器使用其保存的值作为返回地址，回到中断的位置继续执行。

但值得注意的是：

1. mepc 是一个可读可写的寄存器，软件也可以修改该值，达到修改返回后运行的 PC 指针位置的目的。
2. 当发生中断时，即异常原因寄存器 mcause[31]=1 时，mepc 的值更新为中断时下一条未执行的指令 PC 值。

而发生异常时，异常原因寄存器 mcause[31]=0 时，mepc 的值更新为当前异常的指令 PC 值。因此这时异常返回时，如果直接使用 mepc 的值返回，将重新尝试执行异常指令，此时有可能重复进入异常。对于部分异常情况，可以修改 mepc 的值为下一条未执行指令的值后再返回。例如我们因 ecall/ebreak 引起异常，处理异常后，由于 ecall/ebreak (c.ebreak 为 2 字节) 为 4 字节指令，此时只需要软件修改 mepc 的值为 mepc+4 (c.ebreak 为 mepc+2) 后返回即可。

● 更新 mtval

进入异常和中断时，硬件将自动更新 mtval 的值，该值即为引起异常的值。该值一般是：

1. 如果存储器访问引起的异常，硬件会将异常时存储器访问的地址存入 mtval。
2. 如果是非法指令引起的异常，硬件会将该指令的指令编码存入 mtval。
3. 如果是硬件断点引起的异常，硬件会将断点处 PC 值存入 mtval。
4. 对于其他的异常，硬件将 mtval 的值设为 0，例如 ebreak，ecall 指令引起的异常。
5. 进入中断时，硬件将 mtval 的值设为 0。

● 更新 mstatus

进入异常和中断时，硬件会更新 mstatus 中的某些位：

1. MPIE 更新为进入异常和中断前的 MIE 值，异常和中断结束后，MPIE 用于恢复 MIE。
2. MPP 更新为进入异常和中断前的特权模式，异常和中断结束后，MPP 用于恢复之前的特权模式。
3. 青稞 V3 微处理器支持机器模式下的中断嵌套，进入异常和中断后，MIE 根据当前嵌套深度和嵌套使能等信息更新为新的值。

(3) 更新微处理器特权模式

发生异常和中断时，微处理器的特权模式被更新为机器模式。

2.3 异常处理函数

进入异常或中断后，微处理器从 mtvec 寄存器定义的地址和模式执行程序。当使用统一入口时，微处理器从 mtvec[31:2]定义的基地址进入中断。此时异常和中断处理函数中可根据 mcause[31]的值判断引起的是异常或者中断，由异常编码判断异常的类型和原因或者对应的中断，进行相应的处理。

当使用基地址+中断编号*4 进行偏移的方式时，硬件自动根据中断编号跳转至向量表获取异常或者中断函数的入口地址。

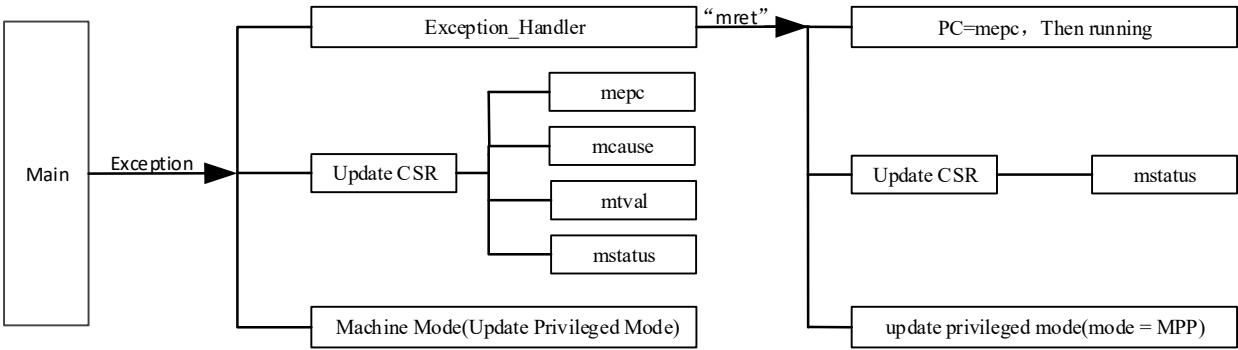
2.4 异常退出

异常或中断处理程序完成之后，需要从服务程序中退出。进入异常和中断后，微处理器由用户模式进入机器模式，异常和中断的处理也在机器模式下完成，当需要退出异常和中断时，需要使用 `mret` 指令进行返回。此时，微处理器硬件将自动执行如下操作：

- PC 指针恢复为 CSR 寄存器 `mepc` 的值，即从 `mepc` 保存的指令地址处开始执行。需要注意异常处理完成后对 `mepc` 的偏移操作。
- 更新 CSR 寄存器 `mstatus`，MIE 恢复为 `MPIE`，MPP 用于恢复之前的微处理器的特权模式。

整个异常的响应过程可由下图 2-1 描述：

图 2-1 异常响应过程示意图



第 3 章 PFIC 与中断控制

青稞 V3 微处理器内部设计了一个可编程快速中断控制器（Programmable Fast Interrupt Controller, PFIC）可管理异常在内的最多 256 个中断。

其中 V3A、V3B、V3C、V3V 前 16 个固定为微处理器内部中断，其余为外部中断，即最大支持扩展 240 个外部中断。其主要特征如下：

- 240 个外部中断，每个中断请求都有独立的触发和屏蔽控制位，有专用的状态位
- 可编程多级中断嵌套，最大嵌套深度 2 级
- 特有免表（Vector Table Free, VTF）中断响应机制，4 路可编程直达中断向量地址

其中 V3F 前 32 个固定为微处理器内部中断，其余为外部中断，即最大支持扩展 224 个外部中断。其主要特征如下：

- 224 个外部中断，每个中断请求都有独立的触发和屏蔽控制位，有专用的状态位
- 可编程多级中断嵌套，最大嵌套深度 2 级
- 特有快速中断进出机制，硬件自动压栈和恢复，最大硬件压栈深度 2 级，无需指令开销
- 特有免表（Vector Table Free, VTF）中断响应机制，4 路可编程直达中断向量地址

注：不同型号的微处理器，中断控制器支持的最大嵌套深度和硬件压栈深度不同，具体可参考表 1-1；

V3A、V3B、V3C、V3V 中断和异常的向量表如下表 3-1 所示。

表 3-1 V3A、V3B、V3C、V3V 异常和中断向量表

编号	优先级	类型	名称	描述
0	-	-	-	-
1	-	-	-	-
2	-5	固定	NMI	不可屏蔽中断
3	-4	固定	EXC	异常中断
4	-	-	-	-
5	-3	固定	ECALL-M	机器模式回调中断
6-7	-	-	-	-
8	-2	固定	ECALL-U	用户模式回调中断
9	-1	固定	BREAKPOINT	断点回调中断
10-11	-	-	-	-
12	0	可编程	SysTick	系统定时器中断
13	-	-	-	-
14	1	可编程	SW	软件中断
15	-	-	-	-
16-255	2-241	可编程	External Interrupt	外部中断 16-255

注：ECALL-M、ECALL-U、BREAKPOINT 均为异常 EXC 的不同类型，为使用方便，V3B/C 中将其独立出来，V3A 中以上三个入口地址和 EXC 共用。

V3F 中断和异常的向量表如下表 3-2 所示。

表 3-2 V3F 异常和中断向量表

编号	优先级	类型	名称	描述
0	-6	固定	RESET	复位中断
1	-	-	-	-

2	-5	固定	NMI	不可屏蔽中断
3	-4	固定	HardFault	异常中断
4	-	-	-	-
5	-3	固定	Ecall-M	机器模式回调中断
6-7	-	-	-	-
8	-2	固定	Ecall-U	用户模式回调中断
9	-1	固定	BreakPoint	断点回调中断
10-11	-	-	-	-
12	0	可编程	SysTick0	系统定时器中断 0
13	1	可编程	SysTick1	系统定时器中断 1
14	2	可编程	SW	软件中断
15	-	-	-	-
16	3	可编程	IPC_CH0	核间通信中断 0
17	4	可编程	IPC_CH1	核间通信中断 1
18	5	可编程	IPC_CH2	核间通信中断 2
19	6	可编程	IPC_CH3	核间通信中断 3
20-27	-	-	-	-
28	7	可编程	HSEM	硬件信号量中断
29-31	-	-	-	-
32-255	8-224	可编程	External Interrupt	外部中断 32-255

3.1 PFIC 寄存器组

表 3-3 PFIC 寄存器

名称	访问地址	访问	描述	复位值
PFIC_ISR _x	0xE000E000 -0xE000E01C	RO	中断使能状态寄存器 x	0x00000000
PFIC_IPR _x	0xE000E020 -0xE000E03C	RO	中断挂起状态寄存器 x	0x00000000
PFIC_ITHRESDR	0xE000E040	RW	中断优先级阈值配置寄存器	0x00000000
PFIC_VTFBADDR	0xE000E044	RW	免表中断基地址寄存器 注：仅对 V3A 有效	0x00000000
PFIC_CFGR	0xE000E048	RW	中断配置寄存器 注：仅对 V3A/F/V 有效	0x00000000
PFIC_GISR	0xE000E04C	RO	中断全局状态寄存器	0x00000002
PFIC_VTFIDR	0xE000E050	RW	VTF 中断 ID 配置寄存器 注：仅对 V3B/C/F/V 有效	0x00000000
PFIC_VTFADDR _x	0xE000E060 -0xE000E06C	RW	VTF 中断 x 偏移地址寄存器	0x00000000
PFIC_IENR _x	0xE000E100 -0xE000E11C	WO	中断使能设置寄存器 x	0x00000000
PFIC_IRER _x	0xE000E180 -0xE000E19C	WO	中断使能清除寄存器 x	0x00000000
PFIC_IPSR _x	0xE000E200	WO	中断挂起设置寄存器 x	0x00000000

	-0xE000E21C			
PFIC_IPRRx	0xE000E280 -0xE000E29C	WO	中断挂起清除寄存器 x	0x00000000
PFIC_IACTRx	0xE000E300 -0xE000E31C	RO	中断激活状态寄存器 x	0x00000000
PFIC_IPRIORx	0xE000E400 -0xE000E43C	RW	中断优先级配置寄存器	0x00000000
PFIC_CSTAR0	0xE000E780	RO	内核状态寄存器 0	0x00000000
PFIC_EENR	0xE000EC80	RW	事件使能寄存器 注：部分芯片支持，具体实现见芯片手册	0xFFFFFFFF
PFIC_EPR	0xE000EC84	RO/RW1Z	事件挂起寄存器 注：部分芯片支持，具体实现见芯片手册	0x00000000
PFIC_EWUPR	0xE000EC88	RO	事件唤醒寄存器 注：部分芯片支持，具体实现见芯片手册	0x00000000
PFIC_SCTLR	0xE000ED10	RW	系统控制寄存器	0x00000000

注：1. NMI、EXC、ECALL-M、ECALL-U、BREAKPOINT 默认使能；

2. ECALL-M、ECALL-U、BREAKPOINT 为 EXC 的一种情况；

3. NMI、EXC、ECALL-M、ECALL-U、BREAKPOINT 支持中断挂起清除和设置操作，不支持中断使能清除和设置操作。

各寄存器描述如下：

中断使能状态和中断挂起状态寄存器（PFIC_ISR<0-7>/PFIC_IPR<0-7>）

名称	访问地址	访问	描述	复位值
PFIC_ISR0	0xE000E000	RO	中断 0-31 使能状态寄存器，共 32 个状态位[n]，表示#n 中断使能状态 注：NMI 和 EXC 默认使能	V3A:0x0000000C V3B/C/F:0x0000032C
PFIC_ISR1	0xE000E004	RO	中断 32-63 使能状态寄存器，共 32 个状态位	0x00000000
...	...	...	...	...
PFIC_ISR7	0xE000E01C	RO	中断 224-255 使能状态寄存器，共 32 个状态位	0x00000000
PFIC_IPR0	0xE000E020	RO	中断 0-31 挂起状态寄存器，共 32 个状态位[n]，表示#n 中断的挂起状态	0x00000000
PFIC_IPR1	0xE000E024	RO	中断 32-63 挂起状态寄存器，共 32 个状态位	0x00000000
...	...	...	...	...
PFIC_IPR7	0xE000E03C	RO	中断 244-255 挂起状态寄存器，共 32 个状态位	0x00000000

两组寄存器用于指示中断的使能状态和中断的挂起状态

中断使能设置和清除寄存器 (PFIC_IENR<0-7>/PFIC_IRER<0-7>)

名称	访问地址	访问	描述	复位值
PFIC_IENR0	0xE000E100	WO	中断 0-31 使能设置寄存器，共 32 个设置位[n]，用于中断#n 使能设置 注：NMI 和 EXC 默认使能	0x00000000
PFIC_IENR1	0xE000E104	WO	中断 32-63 使能设置寄存器，共 32 个设置位	0x00000000
...	...	...	...	...
PFIC_IENR7	0xE000E11C	WO	中断 224-255 使能设置寄存器，共 32 个设置位	0x00000000
—	—	—	—	—
PFIC_IRER0	0xE000E180	WO	中断 0-31 使能清除寄存器，共 32 个清除位[n]，用于中断#n 使能清除 注：NMI 和 EXC 无法操作	0x00000000
PFIC_IRER1	0xE000E184	WO	中断 32-63 使能清除寄存器，共 32 个清除位	0x00000000
...	...	...	...	...
PFIC_IRER7	0xE000E19C	WO	中断 244-255 使能清除寄存器，共 32 个清除位	0x00000000

两组寄存器用于使能和屏蔽相应的中断。

注：在使用寄存器屏蔽任意中断或使用 CSR 寄存器屏蔽全局中断时，追加一条“fence.i”指令，用于内核控制状态和中断使能状态之间的同步。

中断挂起设置和清除寄存器 (PFIC_IPSR<0-7>/PFIC_IPRR<0-7>)

名称	访问地址	访问	描述	复位值
PFIC_IPSR0	0xE000E200	WO	中断 0-31 挂起设置寄存器，共 32 个设置位[n]，用于中断#n 挂起设置	0x00000000
PFIC_IPSR1	0xE000E204	WO	中断 32-63 挂起设置寄存器，共 32 个设置位	0x00000000
...	...	...	...	...
PFIC_IPSR7	0xE000E21C	WO	中断 224-255 挂起设置寄存器，共 32 个设置位	0x00000000
—	—	—	—	—
PFIC_IPRR0	0xE000E280	WO	中断 0-31 挂起清除寄存器，共 32 个清除位[n]，用于中断#n 挂起清除	0x00000000
PFIC_IPRR1	0xE000E284	WO	中断 32-63 挂起清除寄存器，共 32 个清除位	0x00000000
...	...	...	...	...
PFIC_IPRR7	0xE000E29C	WO	中断 244-255 挂起清除寄存器，共 32 个清除位	0x00000000

当微处理器使能某个中断时，可以通过中断挂起寄存器直接设置，触发进入中断。使用中断挂起

清除寄存器，清除挂起触发。

中断激活状态寄存器（PFIC_IACR<0-7>）

名称	访问地址	访问	描述	复位值
PFIC_IACR0	0xE000E300	R0	中断 0-31 激活状态寄存器，共 32 个状态位[n]，表示中断#n 正在执行	0x00000000
PFIC_IACR1	0xE000E304	R0	中断 32-63 激活状态寄存器，共 32 个状态位	0x00000000
...	...	...	...	...
PFIC_IACR7	0xE000E31C	R0	中断 224-255 激活状态寄存器，共 32 个状态位	0x00000000

每个中断都有一个活动状态位，当进入中断时，该位被置起，当 mret 返回后，该位被硬件清除。

中断优先级和优先级阈值寄存器（PFIC_IPR<0-7>/PFIC_ITHRESDR）

名称	访问地址	访问	描述	复位值
PFIC_IPR0	0xE000E400	RW	中断 0 优先级配置： V3A/V3V： [7:4]：优先级控制位 若配置无嵌套，无抢占位 若配置嵌套，bit7 为抢占位 [3:0]：保留，固定为 0 V3B： [7:6]：优先级控制位 若配置无嵌套，无抢占位 若配置嵌套，均为抢占位，但最多允许发生两级中断 [5:0]：保留，固定为 0 V3B/V3C/V3F： [7:5]：优先级控制位 若配置无嵌套，无抢占位 若配置嵌套，均为抢占位，但最多允许发生两级中断 [4:0]：保留，固定为 0 注：优先级数值越小，优先级约高，同一抢占优先级中断若同时挂起，优先执行优先级高的中断。	0x00
PFIC_IPR1	0xE000E401	RW	中断 1 优先级设置，功能同 PFIC_IPR0	0x00
PFIC_IPR2	0xE000E402	RW	中断 2 优先级设置，功能同 PFIC_IPR0	

...	...	...	...	...
PFIC_IPRIOR254	0xE000E4FE	RW	中断 254 优先级设置, 功能同 PFIC_IPRIOR0	0x00
PFIC_IPRIOR255	0xE000E4FF	RW	中断 255 优先级设置, 功能同 PFIC_IPRIOR0	0x00
—	—	—	—	—
PFIC_ITHRESDR	0xE000E040	RW	中断优先级阈值设置 V3A/V3V: [31:8]: 保留, 固定为 0 [7:4]: 优先级阈值 [3:0]: 保留, 固定为 0 V3B: [31:8]: 保留, 固定为 0 [7:6]: 优先级阈值 [5:0]: 保留, 固定为 0 V3C/V3F: [31:8]: 保留, 固定为 0 [7:5]: 优先级阈值 [4:0]: 保留, 固定为 0 注: 优先级值 $\geq$ 阈值的中断, 当发生挂起时不执行中断服务函数, 该寄存器为 0 时, 表示阈值寄存器无效。	0x00

注: 中断嵌套级数和抢占优先级位宽, 抢占优先级位宽配置可能不同, 使用需参考具体芯片手册。

中断配置寄存器 (PFIC_CFGR)

名称	访问地址	访问	描述	复位值
PFIC_CFGR	0xE000E048	RW	中断配置寄存器	0x00000000

对于 V3A 其各位定义为:

位	名称	访问	描述	复位值
[31:16]	KEYCODE	WO	对应不同的目标控制位, 需要同步写入相应的安全访问标识数据才能修改, 读出数据固定为 0。 KEY1 = 0xFA05; KEY2 = 0xBCAF; KEY3 = 0xBEEF。	0
[15:8]	Reserved	RO	保留。	0
7	SYSRESET	WO	系统复位 (同步写入 KEY3)。自动清 0。 写 1 有效, 写 0 无效。	0
6	PFICRESET	WO	PFIC 模块复位, 自动清 0。 写 1 有效, 写 0 无效。	0

5	EXCRESET	WO	异常中断挂起清除（同步写入 KEY2）。 写 1 有效，写 0 无效。	0
4	EXCSET	WO	异常中断挂起设置（同步写入 KEY2）。 写 1 有效，写 0 无效。	0
3	NMIRESET	WO	NMI 中断挂起清除（同步写入 KEY2）。 写 1 有效，写 0 无效。	0
2	NMISSET	WO	NMI 中断挂起设置（同步写入 KEY2）。 写 1 有效，写 0 无效。	0
1	NESTCTRL	RW	中断嵌套使能控制： 1：关闭；0：打开（同步写入 KEY1）。	0
0	HWSTKCTRL	RW	硬件压栈使能控制： 1：关闭；0：打开（同步写入 KEY1）。	0

对于 V3F/V3V 其各位定义为：

位	名称	访问	描述	复位值
[31:0]	SYSRST	WO	系统复位写入 0xBEEF0080 时系统复位，其他值无效。	0

中断全局状态寄存器（PFIC_GISR）

名称	访问地址	访问	描述	复位值
PFIC_GISR	0xE000E04C	RO	中断全局状态寄存器	0x00000000

其各位定义为：

位	名称	访问	描述	复位值
[31:16]	Reserved	RO	保留。	0
[15:14]	EXSTATE	RO	内核状态寄存器，用于获取内核的运行状态： 00：运行； 01：浅睡眠； 10：深睡眠； 11：深睡眠（锁定）。 注：该位仅对 V3B/C/F/V 有效	0
13	LOCKSTA	RO	当前处理器是否处于锁定状态： 1：锁定状态； 0：非锁定状态。 注：该位仅对 V3B/C/F/V 有效	0
12	DBGMODE	RO	当前处理器是否处于调试状态： 1：调试状态； 0：非调试状态。 注：该位仅对 V3B/C/F/V 有效	0
11	GLOBLIE	RO	全局中断使能： 1：使能中断； 0：禁止中断。 注：该位仅对 V3B/C/F/V 有效	0

10	Reserved	R0	保留。	0
9	GPENDSTA	R0	当前是否有中断处于挂起： 1：有； 0：没有。	0
8	GACTSTA	R0	当前是否有中断被执行： 1：有； 0：没有。	0
[7:0]	NESTSTA	R0	当前中断嵌套状态： 0x03：第 2 级中断中； 0x01：第 1 级中断中； 0x00：没有中断发生； 其他：不可能情况。	0

免表中断 ID 基址和偏移地址寄存器 (PFIC_VTFBADDR/PFIC_VTFADDR<0-3>)

名称	访问地址	访问	描述	复位值
PFIC_VTFBADDR	0xE000E044	RW	[31:28]：免表中断的目标地址的高 4 位。 [27:0]：保留。 注：该寄存器仅对 V3A 有效	0x00000000
PFIC_VTFIDR	0xE000E050	RW	[31:24]：免表中断 3 的编号 [23:16]：免表中断 2 的编号 [15:8]：免表中断 1 的编号 [7:0]：免表中断 0 的编号 注：该寄存器仅对 V3B/C/F/V 有效	0x00000000
—	—	—	—	—
PFIC_VTFADDR0	0xE000E060	RW	对于 V3A： [31:24]：免表中断 0 中断编号。 [23:0]：免表中断目标地址低 24 位，其中低 20 位配置有效，[23:20]固定为 0。 对于 V3B/C/F/V： [31:1]：免表中断 0 地址，两字节对齐 [0]： 1：启用免表中断 0 通道 0：关闭	V3A： 0x00000000 V3B/C/F/V： 0xFFFFFFFF
PFIC_VTFADDR1	0xE000E064	RW	对于 V3A： [31:24]：免表中断 1 中断编号。 [23:0]：免表中断目标地址低 24 位，其中低 20 位配置有效，[23:20]固定为 0。 对于 V3B/C/F/V： [31:1]：免表中断 1 地址，	V3A： 0x00000000 V3B/C/F/V： 0xFFFFFFFF

			两字节对齐 [0]: 1: 启用免表中断 1 通道 0: 关闭	
PFIC_VTFADDR2	0xE000E068	RW	对于 V3A: [31:24]: 免表中断 2 中断编号。 [23:0]: 免表中断目标地址低 24 位, 其中低 20 位配置有效, [23:20]固定为 0。 对于 V3B/C/F/V: [31:1]: 免表中断 2 地址, 两字节对齐 [0]: 1: 启用免表中断 2 通道 0: 关闭	V3A: 0x00000000 V3B/C/F/V: 0xFFFFFFFF
PFIC_VTFADDR3	0xE000E06C	RW	对于 V3A: [31:24]: 免表中断 3 中断编号。 [23:0]: 免表中断目标地址低 24 位, 其中低 20 位配置有效, [23:20]固定为 0。 对于 V3B/C/F/V: [31:1]: 免表中断 3 地址, 两字节对齐 [0]: 1: 启用免表中断 3 通道 0: 关闭	V3A: 0x00000000 V3B/C/F/V: 0xFFFFFFFF

内核状态寄存器 0 (PFIC_CSTAR0)

名称	访问地址	访问	描述	复位值
PFIC_CSTAR0	0xE000E780	RO	内核状态寄存器 0	0x00000000

其各位定义为:

位	名称	访问	描述	复位值
[31:16]	Reserved	RO	保留。	0
[15:14]	CPU_ex_state_0	RO	内核 C0 状态寄存器, 用于获取内核 C0 的运行状态: 00: 运行; 01: 浅睡眠; 10: 深睡眠; 11: 深睡眠(锁定)。	0
13	CPU_lock_up_0	RO	内核 C0 锁定状态寄存器, 用于查询内核	0

			C0 是否处于锁定态： 1：锁定； 0：非锁定。	
12	CPU_dbg_mode_0	R0	内核 C0 调试模式寄存器，用于查询内核 C0 是否处于调试模式： 1：内核处于调试模式； 0：内核处于非调试模式。	0
11	CPU_globl_ie_0	R0	内核 C0 全局中断使能寄存器，用于查询内核 C0 全局中断是否使能： 1：内核全局中断使能开启； 0：内核全局中断使能关闭。	0
10	Reserved	R0	保留。	0
9	CPU_irq_pend_0	R0	内核 C0 中断挂起标志寄存器，用于查询内核 C0 是否存在未处理的中断： 1：内核存在未处理的中断； 0：内核不存在未处理的中断。	0
8	CPU_irq_active_0	R0	内核 C0 中断活跃标志寄存器，用于查询内核 C0 是否正在处理中断 1：内核正在处理中断； 0：内核未处理中断。	0
[7:0]	CPU_nest_sta_0	R0	内核 C0 中断嵌套状态寄存器，用于查询内核 C0 中断的嵌套状态。同 CSR 寄存器的 INEST_CTLR 寄存器 nest_sta 位。	0

事件使能寄存器 (PFIC_EENR)

名称	访问地址	访问	描述	复位值
PFIC_EENR	0xE000EC80	RW	事件使能寄存器	0x00000000

部分芯片支持，具体实现见芯片手册，其各位定义为：

位	名称	访问	描述	复位值
[31:0]	EVENTEN	RW	31#-0#事件唤醒使能： 1：允许对应事件在睡眠中唤醒内核； 0：屏蔽对应事件，本事件发生时不允许唤醒内核。 最多支持 32 个事件源，其中 0-7 为电平触发（高有效），8-31 为脉冲触发： 0#：内核暂停请求； 1#：内核中断请求； 7#-2#：保留； 8#：内核进入中断； 9#：内核退出中断； 10#：新的中断挂起；（对于共享中断，不区分该中断是否路由至本地内核） 11#：内核发送事件；（本地内核的发送事件）	0xFFFFFFFF FFF

			12#：内核接收事件；（其它内核的发送事件） 13#：系统接收事件；（系统配置的事件，例如 GPIO 翻转事件...） 31#-14#：保留。	
--	--	--	---	--

事件挂起寄存器（PFIC_EPR）

名称	访问地址	访问	描述	复位值
PFIC_EPR	0xE000EC84	RW	事件挂起寄存器	0x00000000

部分芯片支持，具体实现见芯片手册，其各位定义为：

位	名称	访问	描述	复位值
[31:8]	EVENT_PEND	RW1Z	31#-8#事件挂起状态，写 1 可清零： 1：对应事件发生未被处理； 0：对应事件未发生。	0
[7:0]	EVENT_PEND	RO	7#-0#事件挂起状态，不可清除： 1：对应事件发生未被处理； 0：对应事件未发生。	0

事件唤醒寄存器（PFIC_EWUPR）

名称	访问地址	访问	描述	复位值
PFIC_EWUPR	0xE000EC88	RO	事件唤醒寄存器	0x00000000

部分芯片支持，具体实现见芯片手册，其各位定义为：

位	名称	访问	描述	复位值
[31:0]	EVENT_WUP	RO	31#-0#事件唤醒寄存器： 1：对应事件将内核从睡眠中唤醒； 0：对应事件不是本次唤醒内核的事件。	0

系统控制寄存器（PFIC_SCTLR）

名称	访问地址	访问	描述	复位值
PFIC_SCTLR	0xE000ED10	RW	系统控制寄存器	0x00000000

其各位定义如下：

位	名称	访问	描述	复位值
31	SYSRESET	WO	系统复位，自动清 0。写 1 有效，写 0 无效。 注：该位仅对 V3B/C/F/V 有效	0
[30:24]	Reserved	RO	保留。	0
[23:16]	HART_ID	RO	内核 ID 寄存器，用于获取读本寄存器的内核 ID： 1：本内核为 C1； 0：本内核为 C0。 注：该位仅对 V3F 有效	0

[15:6]	Reserved	RO	保留。	0
5	SETEVENT	WO	设置事件，可以唤醒 WFE 的情况。	0
4	SEVONPEND	RW	当发生事件或者中断挂起状态时，可以从 WFE 指令后唤醒系统，如果未执行 WFE 指令，将在下次执行该指令后立即唤醒系统。 1: 启用的事件和所有中断（包括未开启中断）都能唤醒系统； 0: 只有启用的事件和启用的中断可以唤醒系统。	0
3	WFI TOWFE	RW	将 WFI 指令当成是 WFE 执行。 1: 将之后的 WFI 指令当做 WFE 指令； 0: 无作用。	0
2	SLEEPDEEP	RW	控制系统的低功耗模式： 1: deepsleep 0: sleep	0
1	SLEEPONEXIT	RW	控制离开中断服务程序后，系统状态： 1: 系统进入低功耗模式； 0: 系统进入主程序。	0
0	Reserved	RO	保留。	0

3.2 中断相关 CSR 寄存器

另外，下列 CSR 寄存器也对中断的处理有重大影响：

中断系统控制寄存器（intsyscr）

该寄存器仅对 V3A 无效：

名称	CSR 地址	访问	描述	复位值
intsyscr	0x804	URW	中断系统控制寄存器	0x0000E002

其各位定义为：

位	名称	访问	描述	复位值
31	LOCK	URO	0: 该寄存器用户模式下可读写； 1: 该寄存器仅机器模式下可读写。 注：该配置位 1.0 以后版本有效。	0
[30:6]	Reserved	URO	保留。	0x380
5	GIHWSTKNEN	URW1	全局中断和硬件压栈关闭使能。 注：该位常使用于实时操作系统中，中断切换上下文时，置位该位，可关闭全局中断和硬件压栈出栈，当上下文切换完成，执行完中断返回后，硬件自动清除该位。	0
4	Reserved	URO	保留	0
[3:2]	PMTCFG	URW	优先级抢占位个数配置： 00: 抢占位个数为 0； 01: 抢占位个数为 1；	0

			10: 抢占位个数为 2; 11: 抢占位个数为 3。 注: 该配置位 1.0 以后版本有效。	
1	INESTEN	URW	中断嵌套功能使能, 固定值为 1: 0: 关闭; 1: 使能。 注: 1. 实际嵌套级数由 CSR 0xBC1 中 NEST_LVL 控制; 2. 仅 1.0 以后版本可写。	1
0	HWSTKEN	URW	硬件压栈使能: 0: 硬件压栈功能关闭; 1: 硬件压栈功能使能。	0

机器模式异常基地址寄存器 (mtvec)

名称	CSR 地址	访问	描述	复位值
mtvec	0x305	MRW	异常基地址寄存器	0x00000000

其各位定义为:

位	名称	访问	描述	复位值
[31:2]	BASEADDR	MRW	中断向量表基地址, 其中位[9:2]固定为 0。	0
1	MODE1	MRW	中断向量表识别模式: 0: 按跳转指令识别, 有限范围, 支持非跳指令; 1: 按绝对地址识别, 支持全范围, 但必须跳转。 注: 该位仅对 V3B/C/F/V 有效	0
0	MODE0	MRW	中断或异常入口地址模式选择: 0: 使用统一入口地址; 1: 根据中断编号*4 进行地址偏移。	0

对于 V3 系列微处理器的 MCU, 启动文件里默认配置了 MODE0 为 1, 异常或中断的入口根据中断编号*4 进行偏移, 注意 V3A 微处理器向量表处存储的是一条跳转指令, 而 V3B/C/F/V 微处理器既可以是一条跳转指令也可以使用中断函数的绝对地址, 默认启动文件中将其配置为绝对地址。

微处理器配置寄存器 (corecfr)

该寄存器对 V3A 无效:

名称	CSR 地址	访问	描述	复位值
corecfr	0xBC0	MRW	微处理器配置寄存器	V3B/C: 0x00000001 V3F: 0x12370000

其各位定义为:

位	名称	访问	描述	复位值
[31:28]	FADD_CLKDIV	MRW	浮点加指令分频系数, 默认值为 1, 即浮点加指令频率为主频的 2 分频。 注: 该位仅对 V3F 有效。	0x1

[27:24]	FMUL_CLKDIV	MRW	浮点乘指令分频系数，默认值为 2，即浮点乘指令频率为主频的 3 分频。 注：该位仅对 V3F 有效。	0x2
[23:20]	FMAC_CLKDIV	MRW	浮点乘加指令分频系数，默认值为 3，即浮点乘加指令频率为主频的 4 分频。 注：该位仅对 V3F 有效。	0x3
[19:16]	FDIV_CLKDIV	MRW	浮点除指令分频系数，默认值为 7，即浮点除指令频率为主频的 8 分频。 注：该位仅对 V3F 有效。	0x7
7	CSTA_FAULT_IE	MRW	内核状态错误中断使能： 0：状态错误时，不产生 NMI 中断； 1：状态错误时，产生 NMI 中断。	0
6	INT_FENCE	MRW	FENCE 屏蔽中断使能： 0：执行 fence 指令时不清除中断请求； 1：执行 fence 指令时清除中断请求。	0
5	IE_REMAP_EN	MRW	MIE 寄存器映射使能： 0：CSR 地址 0x800 为只读寄存器，返回值为 MSTATUS 的值； 1：CSR 地址 0x800 的位 3 和位 7 分别映射为 MSTATUS 寄存器的位 MIE 和 MSTATUS 寄存器的位 MPIE；	0
4	Reserved	MRO	保留。	0
3	ROM_LOOP_ACC	MRW	ROM 区指令循环体加速使能： 0：关闭 ROM 区循环加速功能； 1：循环体在 128 字节以内的连续指令将被全部加速，而 256 字节以内的连续指令将被部分加速； 注：该位对 V3A/V3F 无效。	0
2	ROM_JUMP_ACC	MRW	ROM 区指令跳转加速使能： 0：关闭 ROM 区指令跳转加速； 1：使能 ROM 区指令跳转加速。 注：该位对 V3A/V3F 无效。	0
[1:0]	FETCH_MODE	MRW	取指模式： 00：预取关闭。指令预取功能关闭，避免无效的取指操作，CPU 流水线上至多存在 1 条有效指令。此模式功耗最低，性能下降约 2~3 倍。 01：预取模式 1。指令预取功能打开，CPU 将持续对指令存储器进行访问，直到内部指令缓冲器待执行指令数量超过一定数量，或者指令缓冲器已满时，暂停取指令；（CPU 预测失败将引入冗余取指操作，在部分情况下，执行单元将额外引入 0~2 个周期的气泡，大部分程序性能下降不明显）； 其他：保留。	0x1

浮点算数指令为多周期指令，对于不同类型的浮点指令，最高运行频率约束查看各芯片手册。根据处理器的运行频率，用户需配置浮点运算分频系数，使分频后的频率不高于上表所示的最大运行频率。

$$f_{xxx_freq(max)} \geq \text{内核主频} / f_{xxx_clkdiv}$$

中断嵌套控制寄存器（inestcr）

该寄存器对 V3A 无效：

名称	CSR 地址	访问	描述	复位值
inestcr	0xBC1	MRW	中断嵌套控制寄存器	0x00000000

其各位定义为：

位	名称	访问	描述	复位值
31	Reserved	MRO	保留。	0
30	NEST_OV	MRW	中断/异常嵌套溢出标志位，写 1 清零： 0：中断未溢出； 1：中断溢出标志。 注：中断溢出仅会在执行二级中断服务函数产生指令异常或 NMI 中断时发生。此时异常和 NMI 中断正常进入，但是 CPU 堆栈溢出，不可从此异常和 NMI 中断退出。	0
[29:12]	Reserved	MRO	保留。	0
[11:8]	NEST_STA	MRO	嵌套状态标志位： 0000：无中断； 0001：一级中断； 0011：二级中断（一级嵌套）； 0111：三级中断（溢出）； 1111：四级中断（溢出）。	0
[7:2]	Reserved	MRO	保留。	0
[1:0]	NEST_LVL	MRW	嵌套等级： 00：禁止嵌套，关闭嵌套功能； 01：一级嵌套，打开嵌套功能； 其他：无效。 注：对该域写 10 或 11，该域被置为 01，对该与写 11 时，读该寄存器可获得芯片最高嵌套等级。	0

用户模式全局中断使能寄存器（gintenr）

该寄存器对 V3A 无效：

名称	CSR 地址	访问	描述	复位值
gintenr	0x800	URW	全局中断使能寄存器	0x00000000

该寄存器用于控制全局中断的使能与屏蔽，机器模式下全局中断的使能与屏蔽可以通过寄存器 mstatus 中的 MIE，MPIE 位控制，该寄存器用户模式下无法操作。而全局中断使能寄存器 gintenr 是 mstatus 中 MIE 和 MPIE 的映射，用户模式下可以通过操作 gintenr，用于 MIE 和 MPIE 的置位和清零。

其各位定义为：

位	名称	访问	描述	复位值
31	SD	RO	用于标记浮点指令是否处于脏状态。 注：该位仅适用于 V3F。	0
[30:15]	Reserved	URO	保留。	0
[14:13]	FS	RO	浮点指令集状态，值不为 0 时允许单精度浮点指令(F)执行： 00: OFF; 01: Initial; 10: Clean; 11: Dirty。 注：该位仅适用于 V3F。	0
[12:11]	MPP	URO	进中断前特权模式。	0
[10:8]	Reserved	URO	保留。	0
7	MPIE	URW	当 0xBC0 (CSR) bit5 位使能时，该位在用户模式下可读可写。	0
[6:4]	Reserved	URO	保留。	0
3	MIE	URW	当 0xBC0 (CSR) bit5 位使能时，该位在用户模式下可读可写。	0
[2:0]	Reserved	URO	保留。	0

3.3 中断嵌套

配合中断配置寄存器 PFIC_CFGR 和中断优先级寄存器 PFIC_IPRIOR，可以允许中断发生嵌套。在中断配置寄存器中使能嵌套（V3 系列微处理器默认打开嵌套），并且配置对应中断的优先级。优先级数值越小，优先级越高。抢占位数值越小，抢占优先级越高。同一抢占优先级下，若有中断同时挂起，微处理器优先响应优先级数值较小（优先级高）的中断。

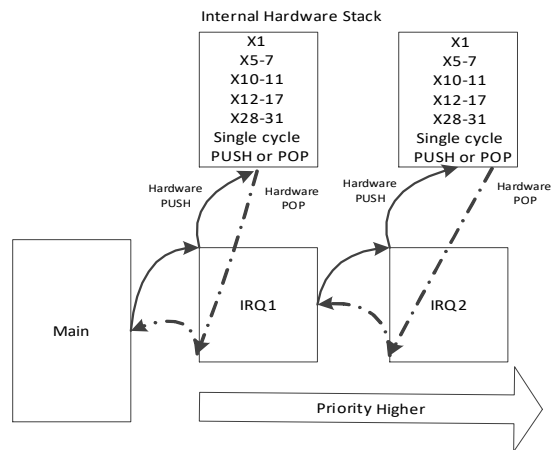
3.4 硬件压栈

当发生异常或者中断时，微处理器停止当前程序流，转向执行异常或者中断处理函数时，需要对当前程序流的现场进行保存。异常或中断返回后，需要恢复现场后，继续执行停止的程序流。对于 V3 系列微处理器，这里的“现场”指的是表 1-2 中，所有的 Caller Saved 寄存器。

V3 系列微处理器，支持硬件单周期自动保存其中 16 个整型 Caller Saved 寄存器至内部的堆栈区域，该区域对用户不可见。当异常或者中断返回后，硬件单周期自动从内部堆栈区恢复数据至 16 个整型寄存器。硬件压栈支持嵌套，嵌套深度最大为 2 级。

微处理器压栈示意图如下图所示：

图 3-1 压栈示意图



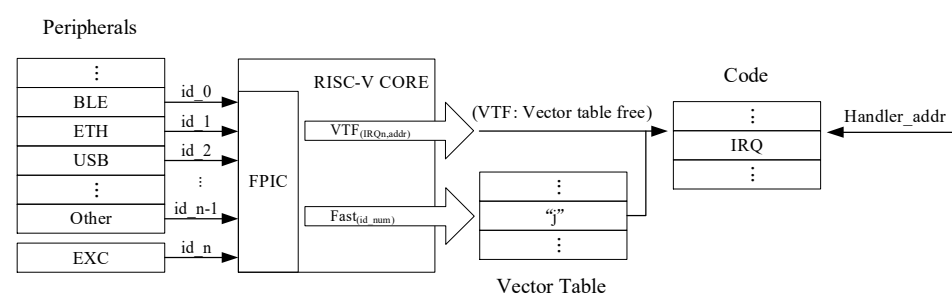
注： 1. 使用硬件压栈的中断函数需要使用 `MRS` 或者其提供的工具链进行编译且中断函数需要采用 `__attribute__((interrupt("WCH-Interrupt-fast")))` 声明。
2. 使用软件压栈的中断函数采用 `__attribute__((interrupt()))` 声明。

3.5 免表中断

可编程快速中断控制器（PFIC）提供四个免表（Vector Table Free）中断通道，即不经过中断向量表的查表过程，直达中断函数入口。

正常配置一个中断函数的同时，将其中断编号、中断服务函数基地址和偏移地址写入对应的 PFIC 控制器寄存器中，即可使能 VTF 通道。其可以减小中断响应延迟，在非零等待存储器中更明显。PFIC 对于快速中断和免表中断的响应过程如下图 3-2 所示。

图 3-2 可编程快速中断控制器示意图



第 4 章 物理内存保护 PMP

为了提高系统安全，青稞 V3 系列微处理器按照 RISC-V 架构标准设计了物理内存保护（Physical Memory Protection, PMP）模块。最多支持 4 个物理区域（region）的访问权限管理。权限包括读（R）、写（W）、执行（X）属性，保护区域的长度最小可设置为 4 字节。PMP 模块在用户模式下一直生效，而在机器模式下可通过锁定（L）属性，可选生效。

如果访问违当前权限限制，将会触发产生异常中断。PMP 模块包括 4 组 8bit 的配置寄存器（一组 32bit）和 4 组地址寄存器，所有的寄存器需要使用 CSR 指令在机器模式下访问。

注：不同型号微处理器 PMP 支持的保护区域个数可能不同，*pmpcfg<n>*和 *pmpaddr<i>*寄存器支持的个数也有不同，详见表 1-1。

4.1 PMP 寄存器组

V3 微处理器 PMP 模块支持的 CSR 寄存器列表如下表 4-1 所示。

表 4-1 PMP 模块寄存器组

名称	CSR 地址	访问	描述	复位值
pmpcfg0	0x3A0	MRW	PMP 配置寄存器 0	0x00000000
pmpaddr0	0x3B0	MRW	PMP 地址寄存器 0	0xFFFFFFFF
pmpaddr1	0x3B1	MRW	PMP 地址寄存器 1	0xFFFFFFFF
pmpaddr2	0x3B2	MRW	PMP 地址寄存器 2	0xFFFFFFFF
pmpaddr3	0x3B3	MRW	PMP 地址寄存器 3	0xFFFFFFFF

4.2 pmp<i>cfg 寄存器

pmpcfg<n>，为 PMP 单元的配置寄存器，每个寄存器含有四个 8 位的 *pmp<i>cfg* 域，对应四个区域的配置，*pmp<i>cfg* 表示区域 *i* 的配置值。其格式如下表 4-2 所示。

表 4-2 pmpcfg0 寄存器

31	24	23	16	15	8	7	0									
pmp3cfg				pmp2cfg				pmp1cfg				pmp0cfg				pmpcfg0

pmp<i>cfg 用于配置区域 *i*，其位定义具体描述如下表 4-3 所示。

表 4-3 pmp<i>cfg 寄存器

位	名称	描述
7	L	锁定使能，机器模式下可解锁 0：不锁定； 1：锁定相关寄存器。
[6:5]	—	保留。
[4:3]	A	地址对齐及保护区域范围选择。 00：OFF（PMP 关闭） 01：TOR（顶端对齐保护） 10：NA4（固定四字节保护） 11：NAPOT（ $2^{(G+2)}$ 字节保护， $G \geq 1$ ）
2	X	可执行属性。 0：无执行权限 1：有执行权限
1	W	可写入属性。 0：无写入权限 1：有写入权限

0	R	可读出属性 0：无读出权限 1：有读出权限
---	---	-----------------------------

4.3 pmpaddr<i>寄存器

pmpaddr<i>寄存器用于配置区域 i 的地址，标准定义在 RV32 架构下，其是对一个 34 位物理地址的高 32 位的编码，其格式如下表 4-4 所示。V3 微处理器整个物理地址空间为 4G，故该寄存器的高两位并不使用。

表 4-4 pmpaddr<i>寄存器

31	0
address[33:2]	

当选择 NAPOT 时，地址寄存器的低位还用来指示当前保护区的大小，如下表所示，其中“y”为寄存器的某个位。

表 4-5 PMP 配置和地址寄存器与保护区关系表

pmpaddr	pmpcfg. A	匹配基地址及大小
yyyy…yyyy	NA4	以“yy…yyy00”为基地址，4 字节区域保护
yyyy…yyy0	NAPOT	以“yy…yyy000”为基地址，8 字节区域保护
yyyy…yy01	NAPOT	以“yy…yy0000”为基地址，16 字节区域保护
yyyy…y011	NAPOT	以“yy…y00000”为基地址，32 字节区域保护
…	…	…
yyy01…111	NAPOT	以“y0…000000”为基地址，2 ³¹ 字节区域保护
yy011…111	NAPOT	对整个 2 ³² 字节区域进行保护

4.4 保护机制

pmp<i>cfg 中的 X/W/R 用于设置区域 i 的保护权限，违反相关权限将引起相应的异常：

- (1) 当试图在没有执行权限的 PMP 区域内取指令，将引起取指令访问错误异常（mcause=1）。
- (2) 当试图在没有写入权限的 PMP 区域内写入数据，将引起 store 指令访存错误异常（mcause=7）。
- (3) 当试图在没有读出权限的 PMP 区域内读取数据，将引起 load 指令访存错误异常（mcause=5）。

pmp<i>cfg 中的 A 用于设置区域 i 的保护范围和地址对齐方式，对于 A_ADDR≤region<i><B_ADDR 区域进行内存保护（要求 A_ADDR 和 B_ADDR 均为 4 字节对齐）：

- (1) 如果 B_ADDR - A_ADDR == 2²，则采用 NA4 方式；
- (2) 如果 B_ADDR - A_ADDR == 2^(G+2)，G≥1，且 A_ADDR 为 2^(G+2) 对齐则采用 NAPOT 方式；
- (3) 否则采用 TOP 方式。

表 4-6 PMP 地址匹配方式

A 值	名称	描述
0b00	OFF	没有区域要保护
0b01	TOR	顶端对齐区域保护： pmp<i>cfg 下，pmpaddr _{i-1} ≤ region<i><pmpaddr _i ； pmpaddr _{i-1} = A_ADDR >> 2； pmpaddr _i = B_ADDR >> 2。 注：若 PMP 的区域 0 配置为 TOR 方式（i=0），则保护区的下边界为 0 地址，即 0 ≤ addr < pmpaddr ₀ ，均在匹配范围内。
0b10	NA4	固定 4 字节区域保护： pmp<i>cfg 下，以 pmpaddr _i 为起始地址的四个字节

		$pmpaddr_i = A_ADDR \gg 2。$
0b11	NAPOT	保护 $2^{(G+2)}$ 区域， $G \geq 1$ ，此时 A_ADDR 为 $2^{(G+2)}$ 对齐。 $pmpaddr_i = ((A_ADDR (2^{(G+2)} - 1)) \& \sim(1 \ll (G+1))) \gg 2。$

pmp<i>cfg 中的 L 位用于将 PMP 入口锁定, 锁定后配置寄存器 pmp<i>cfg 和地址寄存器 pmpaddr<i>将无法被修改。如果 pmp<i>cfg 中 A 被设为了 TOR 模式, pmpaddr<i-1>也将无法修改。L 被置位后, pmp<i>cfg 定义的 X/W/R 权限对机器模式也生效, L 清除后, X/W/R 仅对用户模式有效, L 仅在系统复位后清除。

青稞 V3 系列微处理器支持多个区域的保护, 当同一个操作同时匹配多个区域时, 优先匹配编号较小的区域。

第 5 章 系统定时器 SysTick

青稞 V3 系列微处理器内部设计了一个 32 或 64 位计数器 (SysTick)，其时钟源为系统时钟或其 8 分频，CH32V103 芯片中的 V3A 内核仅支持 8 分频。可为实时操作系统提供时基，提供定时、测量时间等。定时器涉及的寄存器型号不同，映射地址不同，具体如下表 5-1 和表 5-2 所示。

表 5-1 V3A SysTick 寄存器列表

名称	访问地址	描述	复位值
STK_CTLR	0xE000F000	系统计数控制寄存器	0x00000000
STK_CNTL	0xE000F004	系统计数器低位寄存器	0xFFFFFFFF
STK_CNTH	0xE000F008	系统计数器高位寄存器 注：仅对 V3A 有效	0xFFFFFFFF
STK_CMPLR	0xE000F00C	计数比较值低位寄存器	0xFFFFFFFF
STK_CMPHR	0xE000F010	计数比较值高位寄存器 注：仅对 V3A 有效	0xFFFFFFFF
STK_SR	0xE000F014	系统计数器状态寄存器	0x00000000

表 5-2 V3B/V3C/V3F/V3V SysTick 寄存器列表

名称	访问地址	描述	复位值
STK_CTLR	0xE000F000	系统计数控制寄存器	0x00000000
STK_SR	0xE000F004	系统计数器状态寄存器	0x00000000
STK_CNTL	0xE000F008	系统计数器低位寄存器	0xFFFFFFFF
STK_CMPLR	0xE000F010	计数比较值低位寄存器	0xFFFFFFFF

各寄存器详细描述如下：

系统计数控制寄存器 (STK_CTLR)

该寄存器适用于 V3A/B/C/F/V：

位	名称	访问	描述	复位值
[31:7]	Reserved	R0	保留。	0
6	CID	RW	计数器分配内核寄存器，用于配置计数器向指定内核产生中断请求，在计数器使能被配置有效时自动更新为操作的内核 ID。 注：详见具体芯片。	0
5	INIT	RW1	计数器初始值更新： 1：向上计数时更新为 0，向下计数时更新为比较值； 0：无效。 注：对 V3A 无效	0
4	MODE	RW	计数模式： 1：向下计数； 0：向上计数。 注：对 V3A 无效	0
3	STRE	RW	自动重载计数使能位： 1：向上计数到比较值后重新从 0 开始计数，向下计数到 0 后，重新从比较值开始计数； 0：继续向上/向下计数。 注：对 V3A 无效	0

2	STCLK	RW	计数器时钟源选择位： 1: HCLK 做时基； 0: HCLK/8 做时基。	0
1	STIE	RW	计数器中断使能控制位： 1: 使能计数器中断； 0: 关闭计数器中断。	0
0	STE	RW	系统计数器使能控制位： 1: 启动系统计数器 STK； 0: 关闭系统计数器 STK，计数器停止计数。	0

系统计数器状态寄存器 (STK_SR)

该寄存器适用于 V3B/C/F/V:

位	名称	访问	描述	复位值
31	SWIE	RW	软件中断触发使能 (SW) : 1: 触发软件中断； 0: 关闭触发。 注: 进入软件中断后须将此位清 0, 否则将一直触发	0
[30:1]	Reserved	RO	保留	0
0	STK_ISR0	RW	Systick 中断标志: 1: 定时器存在未处理的中断请求； 0: 定时器不存在中断请求。	0

注: 不同型号芯片中可能实现了多路系统计数器, 具体中断标志位宽参见芯片手册。

该寄存器适用于 V3A:

位	名称	访问	描述	复位值
[31:2]	Reserved	RO	保留	0
1	STK_ISR0	RW	Systick 中断标志: 1: 定时器存在未处理的中断请求； 0: 定时器不存在中断请求。 注: 该位不适用于 CH32V103 芯片。	0
0	SWIE	RW	软件中断触发使能 (SWI) : 1: 触发软件中断； 0: 关闭触发。 注: 进入软件中断后须将此位清 0, 否则将一直触发	0

系统计数器低位寄存器 (STK_CNTL)

该寄存器适用于 V3A/B/C/F/V:

位	名称	访问	描述	复位值
[31:0]	CNTL	RW	当前计数器计数值低 32 位。 对于 V3A 此寄存器可按 8 位/16 位/32 位读取, 但是只能以 8 位进行写, 其他型号无限制。 注: CH32V103 芯片仅支持向上计数。	0xFFFFFFFF

注: V3A 中寄存器 STK_CNTL 和寄存器 STK_CNTH 共同构成了 64 位系统计数器。

系统计数器高位寄存器 (STK_CNTH)

该寄存器适用于 V3A:

位	名称	访问	描述	复位值
[31:0]	CNTH	RW	当前计数器计数值高 32 位。 此寄存器可按 8 位/16 位/32 位读取, 但是只能以 8 位进行写。 注: 仅对 V3A 有效, CH32V103 芯片仅支持向上计数。	0xFFFFFFFF

注: V3A 中寄存器 STK_CNTH 和寄存器 STK_CNTH 共同构成了 64 位系统计数器。

计数比较值低位寄存器 (STK_CMPLR)

该寄存器适用于 V3A/B/C/F/V:

位	名称	访问	描述	复位值
[31:0]	CMPL	RW	设置计数器比较值低 32 位。 当向上计数 CMP 寄存器值等于 CNT 计数器值或向下计数 CNT 计数器值为 0 时将触发 STK 中断。 对于 V3A 此寄存器可按 8 位/16 位/32 位读取, 但是只能以 8 位进行写, 其他型号无限制。	0xFFFFFFFF

注: V3A 中寄存器 STK_CMPLR 和寄存器 STK_CMPHR 共同构成了 64 位计数器比较值。

计数比较值高位寄存器 (STK_CMPHR)

该寄存器适用于 V3A:

位	名称	访问	描述	复位值
[31:0]	CMPH	RW	设置计数器比较值高 32 位。 当向上计数 CMP 寄存器值等于 CNT 计数器值或向下计数 CNT 计数器值为 0 时将触发 STK 中断。 此寄存器可按 8 位/16 位/32 位读取, 但是只能以 8 位进行写。 注: 仅对 V3A 有效	0xFFFFFFFF

注: V3A 中寄存器 STK_CMPLR 和寄存器 STK_CMPHR 共同构成了 64 位计数器比较值。

第 6 章 处理器低功耗设置

青稞 V3 系列微处理器支持通过 WFI (Wait For Interrupt) 指令进入睡眠状态，实现较低的静态功耗。配合 PFIC 的系统控制寄存器 (PFIC_SCTLR)，能实现多种睡眠模式和 WFE 指令。

6.1 进入睡眠

青稞 V3 系列微处理器可以通过两种方式进入睡眠，即等待中断 (Wait for Interrupt, WFI) 和等待事件 (Wait For Event, WFE)。WFI 方式是指微处理器进入睡眠后，等待中断来唤醒，醒来后进入相应的中断中去执行。WFE 方式是指微处理器进入睡眠后，等待事件来唤醒，醒来后继续执行之前停止的程序流。

标准的 RISC-V 支持 WFI 指令，直接执行 WFI 命令，即可通过 WFI 方式进入睡眠。而对于 WFE 方式，系统控制寄存器 PFIC_SCTLR 中 WFIOWFE 位用于控制将之后的 WFI 指令当做 WFE 处理，实现 WFE 方式进入睡眠。

根据 PFIC_SCTLR 中 SLEEPDEEP 位控制睡眠的深度：

- 若 PFIC_SCTLR 寄存器中的 SLEEPDEEP 清零，微处理器进入睡眠模式，除 SysTick 及部分唤醒逻辑外的内部单元时钟允许被关闭。
- 若 PFIC_SCTLR 寄存器中的 SLEEPDEEP 置位，微处理器进入深度睡眠模式，所有单元时钟均允许被关闭。

当微处理器处于调试模式下时，无法进入任何一种睡眠模式。

6.2 睡眠唤醒

青稞 V3 系列微处理器因 WFI 和 WFE 睡眠后，可采用以下方式唤醒：

- WFI 方式进入睡眠后，可由以下方式唤醒：
 - (1) 微处理器可被中断控制器响应的中断源唤醒，唤醒后，微处理器先执行中断函数。
 - (2) 进入睡眠模式，调试请求可以使微处理器唤醒，进入深度睡眠，调试请求无法唤醒微处理器。
- WFE 方式进入睡眠后，微处理器可被下面的方式唤醒：
 - (1) 内部或外部的事件，此时无需配置中断控制器，唤醒后继续执行程序。
 - (2) 若配置 PFIC_SCTLR 中的 SEVONPEND 位，中断控制器不使能中断下，但产生新的中断挂起信号时（之前产生的挂起信号不生效）也可以使微处理器唤醒，唤醒后需要手动清除相应的中断挂起标志。
 - (3) 进入睡眠模式调试请求可以使微处理器唤醒，进入深度睡眠，调试请求无法唤醒微处理器。

另外，可以通过配置 PFIC_SCTLR 中的 SLEEPONEXIT 位控制微处理器唤醒后的状态：

- SLEEPONEXIT 置位，最后一级中断返回指令 (mret) 将触发 WFI 方式睡眠。
- SLEEPONEXIT 清零，无影响。

搭载 V3 系列微处理器的各类 MCU 产品可根据 PFIC_SCTLR 不同配置，采取不同的睡眠方式，关闭不同的外设及时钟，执行不同的电源管理策略和唤醒方式，实现多种低功耗模式。

第 7 章 调试支持

青稞 V3 系列微处理器内含一个硬件调试模块，支持复杂的调试操作。调试模块可以暂停和恢复微处理器的运行，当微处理器暂停时，调试模块可以通过抽象命令、program buffer 部署指令等方式访问微处理器的 GPRs、CSRs、储存器、外部设备等等。

调试模块遵循 RISC-V External Debug Support Version0.13.2 规范，详细文档可在基金会官方网站下载。

7.1 调试模块

微处理器内部的调试模块，能够执行调试主机下发的调试操作，包括：

- 通过调试接口访问寄存器
- 通过调试接口可以使微处理器复位、暂停、恢复
- 通过调试接口读写存储器、指令寄存器以及外部设备
- 通过调试接口可以部署多条任意指令
- 通过调试接口设置软件断点
- 通过调试接口设置硬件断点
- 支持抽象命令自动执行
- 支持单步调试

注：V3A不支持硬件断点，V3B硬件断点支持指令地址匹配，V3C、V3F硬件断点支持指令地址和数据地址匹配。

调试模块内部寄存器使用7位地址编码，青稞V3系列微处理器内部实现了以下寄存器：

表7-1 调试模块寄存器列表

名称	访问地址	描述
data0	0x04	数据寄存器 0，可用于暂存数据
data1	0x05	数据寄存器 1，可用于暂存数据
dmcontrol	0x10	调试模块控制寄存器
dmstatus	0x11	调试模块状态寄存器
hartinfo	0x12	微处理器状态寄存器
abstractcs	0x16	抽象命令状态寄存器
command	0x17	抽象命令寄存器
abstractauto	0x18	抽象命令自动执行
progbuf0-7	0x20-0x27	指令缓存寄存器 0-7
haltsum0	0x40	暂停状态寄存器
capabi	0x7c	调试接口寄存器
config	0x7d	调试器配置寄存器
config_pre	0x7e	调试器预配置寄存器

调试主机可通过配置dmcontrol寄存器控制微处理器的暂停、恢复、复位等。也可通过command寄存器触发调试模块生成抽象命令。RISC-V标准定义三种抽象命令类型：访问寄存器、快速访问、访问存储器。青稞V3A微处理器仅支持寄存器访问，其他型号支持寄存器和存储器访问，不支持快速访问。可以通过抽象命令实现寄存器（GPRs、CSRs）访问，存储器的连续访问等。

调试模块内部实现了八个指令缓存寄存器progbuf0-7，调试主机可向缓冲区缓存多条指令（可以是压缩指令），可选择执行完抽象命令后继续执行指令缓存寄存器中的指令，也可直接执行缓存的指令。需要注意的是，progbufs中的最后一条指令需要是一条“ebreak”或“c.ebreak”指令。通过抽

象命令和progbufs中缓存的指令，也可实现储存器、外设等的访问。

各寄存器详细描述如下：

数据寄存器 0 (data0)

位	名称	访问	描述	复位值
[31:0]	data0	RW	数据寄存器 0，用于暂存数据	0

数据寄存器 1 (data1)

位	名称	访问	描述	复位值
[31:0]	data1	RW	数据寄存器 1，用于暂存数据	0

调试模块控制寄存器 (dmcontrol)

该寄存器可以控制微处理器的暂停、复位、恢复。调试主机向对应的字段写数据，即可实现暂停 (haltreq)、复位 (ndmreset)、恢复 (resumereq)。各位描述如下：

位	名称	访问	描述	复位值
31	haltreq	W0	暂停选择的微处理器，受 hartsel 控制 0：清除暂停请求 1：发出暂停请求	0
30	resumereq	W1	恢复选择的微处理器，受 hartsel 控制 0：无效 1：恢复当前微处理器 注：写 1 有效，微处理器恢复后，硬件清零	0
29	Reserve	R0	保留	0
28	ackhavereset	W1	0：无效 1：清除微处理器的 havereset 状态位	0
[27:18]	Reserve	R0	保留	0
[17:16]	hartsel	RW	微处理器选择寄存器，固定值为 0。 注：仅对 V3F 有效	0
[15:2]	Reserve	R0	保留	0
1	ndmreset	RW	0：清除复位 1：复位调试模块以外的整个系统	0
0	dmactive	RW	0：复位调试模块 1：调试模块正常工作	0

调试模块状态寄存器 (dmstatus)

该寄存器用于指示调试模块的状态，是一个只读寄存器，各位的描述如下：

位	名称	访问	描述	复位值
[31:23]	Reserve	R0	保留	0
22	impebreak	R0	固定值，值为 1 表示在 progbuf 后隐含一个 ebreak 指令。	V3A:0 V3B/C/F:1
[21:20]	Reserve	R0	保留	0
19	allhavereset	R0	0：无效 1：微处理器复位	0
18	anyhavereset	R0	0：无效	0

			1: 微处理器复位	
17	allresumeack	R0	0: 无效 1: 微处理器已经恢复	0
16	anyresumeack	R0	0: 无效 1: 微处理器已经恢复	0
15	allnoexit	R0	被选中的微处理器不存在, 受 hartsel 控制 0: 微处理器不存在 1: 微处理器可选 注: 仅对 V3F 有效	0
14	anynoexit	R0	被选中的微处理器不存在, 受 hartsel 控制 0: 微处理器不存在 1: 微处理器可选 注: 仅对 V3F 有效	0
13	allavail	R0	0: 无效 1: 微处理器不可用	0
12	anyavail	R0	0: 无效 1: 微处理器不可用	0
11	allrunning	R0	0: 无效 1: 微处理器正在运行	0
10	anyrunning	R0	0: 无效 1: 微处理器正在运行	0
9	allhalted	R0	0: 无效 1: 微处理器处于暂停	0
8	anyhalted	R0	0: 无效 1: 微处理器出暂停	0
7	authenticated	R0	0: 使用调试模块前需要认证 1: 调试模块已经通过认证	0x1
[6:4]	Reserve	R0	保留	0
[3:0]	version	R0	调试系统支持架构版本 0010: V0.13	0x2

微处理器状态寄存器 (hartinfo)

该寄存器用于向调试主机提供微处理器的信息, 是一个只读寄存器, 各个位描述如下:

位	名称	访问	描述	复位值
[31:24]	Reserve	R0	保留	0
[23:20]	nscratch	R0	支持的 dscratch 寄存器数量	0x3
[19:17]	Reserve	R0	保留	0
16	dataaccess	R0	0: 数据寄存器映射为 CSR 地址 1: 数据寄存器映射为存储器地址	0x1
[15:12]	datasize	R0	数据寄存器数量	0x2
[11:0]	dataaddr	R0	数据寄存器 data0 偏移地址, 其基地址为 0xe0000000, 偏移地址以具体读出为准。	0xXXX

抽象命令控制与状态寄存器 (abstractcs)

该寄存器用于指示抽象命令执行的情况, 调试主机可以通过读取该寄存器, 了解上一个抽象命令是否执行完毕, 并且可以检查抽象命令执行过程中是否产生错误以及错误的类型, 该寄存器的详细描述

述如下：

位	名称	访问	描述	复位值
[31:29]	Reserve	R0	保留	0
[28:24]	progbufsize	R0	表示 program buffer 程序缓存寄存器数量	0x8
[23:13]	Reserve	R0	保留	0
12	busy	R0	0: 无抽象命令执行 1: 有抽象命令正在执行 注: 执行完毕后, 硬件清零	0
11	Reserve	R0	保留	0
[10:8]	cmdr	RW	抽象命令错误类型 000: 无错误 001: 抽象命令执行时对 command、abstractcs、abstractauto 寄存器进行写或者对 data 和 progbuf 寄存器进行读写操作 010: 不支持当前抽象命令 011: 执行抽象命令出现异常 100: 微处理器未暂停或不可用, 而不能执行抽象命令 101: 总线出错 110: 通讯时奇偶校验位错误 111: 其他错误 注: 对应位写 1 用于清零	0
[7:4]	Reserve	R0	保留	0
[3:0]	datacount	R0	数据寄存器数量	0x2

抽象命令寄存器 (command)

调试主机可以通过抽象命令寄存器中写入不同的配置值, 对微处理器内部的 GPRs、CSRs 寄存器进行访问, 以及存储器访问。

当对寄存器访问时, command 寄存器各位定义如下:

位	名称	访问	描述	复位值
[31:24]	cmdtype	WO	抽象命令类型 0: 访问寄存器 1: 快速访问 (未支持) 2: 访问存储器	0
23	Reserve	WO	保留	0
[22:20]	aarsize	WO	访问寄存器数据位宽 000: 8 位 001: 16 位 010: 32 位 011: 64 位 (未支持) 100: 128 位 (未支持) 注: 当访问浮点寄存器 FPRs 时, 仅支持 32 位访问	0
19	aarpostincrement	WO	0: 无影响 1: 访问寄存器后自动增加 regno 的值	0
18	postexec	WO	0: 无影响	0

			1: 执行抽象命令后执行 progbuf 中的命令	
17	transfer	WO	0: 不执行 write 指定的操作 1: 执行 write 指令的操纵	0
16	write	WO	0: 从指定的寄存器拷贝数据至 data0 1: 从 data0 寄存器拷贝数据至指定的寄存器	0
[15:0]	regno	WO	指定访问寄存器 0x0000-0x0fff 为 CSRs 0x1000-0x101f 为 GPRs 0x1020-0x103f 为 FPRs	0

当访问的存储器时，command 寄存器各位定义如下：

位	名称	访问	描述	复位值
[31:24]	cmdtype	WO	抽象命令类型 0: 访问寄存器 1: 快速访问（未支持） 2: 访问存储器	0
23	aamvirtual	WO	0: 访问物理地址 1: 访问虚拟地址	0
[22:20]	aamsize	WO	访问存储器数据位宽 000: 8 位 001: 16 位 010: 32 位 011: 64 位（未支持） 100: 128 位（未支持）	0
19	aampostincrement	WO	0: 无影响 1: 访问存储器成功后将 data1 寄存器存放的地址按照 aamsize 配置的位宽对应的字节数递增 aamsize=0, 按字节访问, data1 加 1 aamsize=1, 按半字范围, data1 加 2 aamsize=2, 按字访问, data1 加 4	0
18	postexec	WO	0: 无影响 1: 执行抽象命令后执行 progbuf 中的命令	0
17	Reserve	RO	保留	0
16	write	WO	0: 从 data1 指定的地址读数据到 data0 1: 向 data1 指定的地址写入 data0 中的数据	0
[15:14]	target-specific	WO	读写方式定义 对于写: 00、01: 直接对存储器进行写 10: 将 data0 中数据与存储器中数据位或后, 将结果写入存储器（仅支持字访问） 11: 将 data0 中数据与存储器中数据位与后, 将结果写入存储器（仅支持字访问） 对于读: 00、01、10、11: 直接对存储器进行读	0

[13:0]	Reserve	R0	保留	0
--------	---------	----	----	---

抽象命令自动执行寄存器 (abstractauto)

该寄存器用于配置调试模块，当对调试模块的 progbufx 和 datax 进行读写时，抽象命令可以再次被执行，该寄存器的描述如下：

位	名称	访问	描述	复位值
[31:16]	autoexecprogbuf	RW	若某位置 1，相应对 progbufx 的读写将导致 command 寄存器中抽象命令再次被执行 注：V3 系列设计 8 个 progbuf，对应位 [23:16]	0
[15:12]	Reserve	R0	保留	0
[11:0]	autoexecdata	RW	若某位置 1，相应对 datax 寄存器的读写将导致 command 寄存器中的抽象命令再次被执行 注：V3 系列设计 2 个 data 寄存器，对应位 [1:0]	0

指令缓存寄存器 (progbufx)

该寄存器用于存放任意指令，部署相应的操作，包括 8 个，V3A 型内核在 progbuf 后未隐性包含“ebreak”指令，V3B/C/F 型内核在 progbuf 后隐性包含“ebreak”指令。

位	名称	访问	描述	复位值
[31:0]	progbuf	RW	缓存操作的指令编码，可以包括压缩指令	0

暂停状态寄存器 (haltsum0)

该寄存器用于指示微处理器是否暂停，每一位指示一个微处理器的暂停状态，当只有一个核时，仅用该寄存器最低位表示。

位	名称	访问	描述	复位值
[31:1]	Reserve	R0	保留	0
0	haltsum0	R0	haltsum0 V3F 内核暂停状态 0：微处理器正常运行 1：微处理器停止	0

调试接口寄存器 (capabi)

该寄存器用于获取调试支持的协议，频率等信息。

位	名称	访问	描述	复位值
[31:28]	swd_version	R0	单线接口版本信息	1
[27:24]	twd_version	R0	双线接口版本信息	1
[23:18]	SWD_FREQ	R0	单线接口采样时钟频率，25MHZ	25
[17:16]	dmi_info	R0	调试器支持单/双线通信信息 00：只支持双线 01：只支持单线 11：支持单/双线	0x3
15	SWD&TWD	R0	单线/双线通信均支持标志位 0：只支持单线或双线通信 1：单/双线通信均支持	1
[14:11]	Reserve	R0	保留	0
10	output_cap	R0	调试器允许数据输出	1
[9:0]	Reserve	R0	保留	0

调试接口配置寄存器 (config)

该寄存器用于配置调试器单/双线模式、输出使能、通信速率等信息，本寄存器不支持直接写入，需要先向config_pre寄存器中写入值后，向本寄存器写入配置指令和配置位掩码后本寄存器值被写入并生效。

位	名称	访问	描述	复位值
[31:16]	config_key	W0	写配置指令寄存器，写入 0x5aa5 时写操作生效	0
15	swdmd	RW	单/双线选择寄存器，写入 1 时同步 swdmd_pre 寄存器值 1: 双线通信 0: 单线通信	1
[14:11]	Reserve	R0	保留	0
10	outen	RW	调试器输出使能，置 1 后允许输出数据，写 1 时同步 outen_pre 寄存器值 0: 输出屏蔽 1: 输出使能	0
[9:2]	Reserve	R0	保留	0
[1:0]	psc1x	RW	单线通信码元速率选择寄存器写 11b 时同步 outen_pre 寄存器值 00: 单线码元基本频率为 8M 01: 单线码元基本频率为 4M 10: 单线码元基本频率为 25M 11: 单线码元基本频率为 12.5M	1

注: psc1x 位的 10b 和 11b 是单位码元速率在不同芯片中配置不同，与 capabi.SWD_FREQ 值相关，详见具体芯片手册。

调试接口预配置寄存器 (config_pre)

该寄存器用于预配置调试器单/双线模式、输出使能、通信速率等信息，向本寄存器写入配置指令正确时写操作生效，config_pre 寄存器与 config 寄存器需连续配置，其间不可操作其他寄存器。

位	名称	访问	描述	复位值
[31:16]	config_key	W0	写配置指令寄存器，写入 0x5aa5 时写操作生效，配置指令正确后读本寄存器读出 0x5aa5 表示预配置寄存器写入成功，允许写入配置寄存器，否则读本寄存器值为 0，不可配置 config 寄存器	0
15	swdmd_pre	RW	单/双线预选择寄存器	1
[14:11]	Reserve	R0	保留	0
10	outen_pre	RW	调试器输出预使能	0
[9:2]	Reserve	R0	保留	0
[1:0]	psc1x_pre	RW	单线通信码元速率预选择寄存器	1

除了上述调试模块的寄存器外，调试功能还涉及部分 CSR 寄存器，主要是调试控制和状态寄存器 dcsr 和调试指令指针 dpc，寄存器详细描述如下：

调试控制和状态寄存器 (dcsr)

位	名称	访问	描述	复位值
---	----	----	----	-----

[31:28]	xdebugver	DRO	0000: 不支持外部调试 0100: 支持标准的外部调试 1111: 支持外部调试, 但是不符合规范	0x4
[27:16]	Reserve	DRO	保留	0
15	ebreakm	DRW	0: 机器模式下的 ebreak 指令的行为如特权文档所述 1: 机器模式下的 ebreak 指令能够进入调试模式	0
[14:13]	Reserve	DRO	保留	0
12	ebreaku	DRW	0: 用户模式下的 ebreak 指令的行为如特权文档所述 1: 用户模式下的 ebreak 指令能够进入调试模式	0
11	stepie	DRW	0: 单步调试下禁止中断 1: 单步调试下启用中断	0
10	Reserve	DRO	保留	0
9	stoptime	DRW	0: 调试模式下系统定时器运行 1: 调试模式下系统定时器停止	0
[8:6]	cause	DRO	进入调试的原因 001: 以 ebreak 指令方式进入调试 (优先级为 3) 010: 以 trigger module 形式进入调试 (优先级为 4, 最高) 011: 以暂停请求形式进入调试 (优先级为 1) 100: 以单步调试形式进入调试 (优先级为 0, 最低) 101: 微处理器复位之后直接停止进入调试模式 (优先级为 2) 其他: 保留	0
[5:4]	Reserve	DRO	保留	0
3	nmi_pend	DRO	不可屏蔽中断挂起状态 注: 仅对 V3F 有效	0
2	step	DRW	0: 关闭单步调试 1: 开启单步调试	0
[1:0]	prv	DRW	特权模式 00: 用户模式 01: 监督模式 (未支持) 10: 保留 11: 机器模式 注: 记录进入调试模式时特权模式, 调试器可以修改该值, 以修改退出调试时的特权模式	0

调试模式程序指针 (dpc)

该寄存器用于保存微处理器进入调试模式之后将要执行的下一条指令的地址, 其值根据进入调试的原因不同, 更新的规则也不相同。dpc 寄存器详细描述如下:

位	名称	访问	描述	复位值
---	----	----	----	-----

[31:0]	dpc	DRW	指令地址	0
--------	-----	-----	------	---

寄存器的更新规则如下表所示：

进入调试方式	dpc 更新规则
ebreak	Ebreak 指令的地址
single step	当前指令的下一条指令的指令地址
trigger module	暂不支持
halt request	进入调试时，下一条将要被执行的指令地址

7.2 调试接口

区别于标准的 RISC-V 定义的 JTAG 接口，青稞 V3 系列微处理器采用单/双线调试接口，遵循沁恒调试接口协议。调试接口负责调试主机和调试模块之间的通讯，实现调试主机对调试模块寄存器的读写操作。沁恒设计了 WCH_Link，并开源其原理图和程序二进制文件，可用于所有 RISC-V 架构的微处理器调试。

具体的调试接口协议参考沁恒调试协议手册。

第 8 章 CSR 寄存器列表

RISC-V 架构中定义了一些控制和状态寄存器（Control and Status Register, CSR），用于控制和记录微处理器的运行状态。前文已经介绍到部分 CSR，本章将详细说明青稞 V3 系列微处理器实现的 CSR 寄存器。

8.1 CSR 寄存器列表

表 8-1 微处理器 CSR 寄存器列表

类型	名称	CSR 地址	访问	描述
RISC-V 标准 CSR	mvendorid	0xF11	MRO	芯片制造商 ID 寄存器
	marchid	0xF12	MRO	架构编号寄存器
	mimpid	0xF13	MRO	硬件实现编号寄存器
	mhartid	0xF14	MRO	内核 ID 寄存器
	mstatus	0x300	MRW	状态寄存器
	misa	0x301	MRW	硬件指令集寄存器
	mtvec	0x305	MRW	异常基地址寄存器
	mscratch	0x340	MRW	机器模式暂存寄存器
	mepc	0x341	MRW	异常程序指针寄存器
	mcause	0x342	MRW	异常原因寄存器
	mtval	0x343	MRW	异常值寄存器
	pmpcfg<i>	0x3A0+i	MRW	物理内存保护单元配置寄存器
	pmpaddr<i>	0x3B0+i	MRW	物理内存保护单元地址寄存器
	tselect	0x7A0	MRW	调试触发选择寄存器
	tdata1	0x7A1	MRW	调试触发数据寄存器 1
	tdata2	0x7A2	MRW	调试触发数据寄存器 2
	tinfo	0x7A4	MRO	触发器信息寄存器
	dcsr	0x7B0	DRW	调试控制与状态寄存器
	dpc	0x7B1	DRW	调试模式程序指针寄存器
	dscratch0	0x7B2	DRW	调试模式暂存寄存器 0
	dscratch1	0x7B3	DRW	调试模式暂存寄存器 1
	dbgmcu_0	0x7C0	MRW	调试寄存器 0
	fflags	0x001	RW	浮点累计异常标记寄存器
	frm	0x002	RW	浮点舍入模式寄存器
	fcsr	0x003	RW	浮点控制和状态寄存器
厂商 自定义 CSR	gintenr	0x800	URW	全局中断使能寄存器
	intsyscr	0x804	URW	中断系统控制寄存器
	corecfgr	0xBC0	MRW	微处理器配置寄存器
	inestcr	0xBC1	MRW	中断嵌套控制寄存器

表 8-2 RISC-V3V 微处理器 CSR 寄存器列表

类型	名称	CSR 地址	访问	描述
RISC-V3V CSR	vxsat	0x009	URW	定点饱和状态寄存器
	vxrm	0x00A	URW	定点舍入寄存器
	vcsr	0x00F	URW	向量控制和状态寄存器
	mcount_inhibit	0x320	MRW	计数器屏蔽寄存器
	mcycle	0xB00	MRW	机器周期计数器寄存器
	minstret	0xB02	MRW	机器指令计数器寄存器
	ucycle	0xC00	RO	用户模式周期计数器寄存器

	uinstret	0xC02	RO	用户模式指令计数器寄存器
	vl	0xC20	URO	元素体长度寄存器
	vtype	0xC21	URO	向量数据类型寄存器
	vlenb	0xC22	URO	向量寄存器字节长度寄存器
	vcontrol	0x805	URW	自定义向量控制寄存器
	vppaddr	0x806	URW	自定义向量寄存器压栈基地址寄存器
	vcause	0x808	URO	自定义向量异常源寄存器
	vtval	0x809	URO	自定义向量异常自陷信息寄存器

8.2 RISC-V 标准 CSR 寄存器

芯片制造商 ID 寄存器（marchid）

位	名称	访问	描述	复位值
[31:0]	mvendorid	MRO	芯片制造商 ID，固定值为 0。 注：仅对 V3F 有效	0

架构编号寄存器（marchid）

此寄存器为只读寄存器，用于指示当前微处理器硬件架构编号，其主要由厂商编码、架构编码、系列编码、版本编码构成。其各位定义如下：

位	名称	访问	描述	复位值
31	Reserved	MRO	保留	1
[30:26]	Vender0	MRO	厂商编码 0 固定为字母“W”编码	0x17
[25:21]	Vender1	MRO	厂商编码 1 固定为字母“C”编码	0x03
[20:16]	Vender2	MRO	厂商编码 2 固定为字母“H”编码	0x08
15	Reserved	MRO	保留	1
[14:10]	Arch	MRO	架构编码 RISC-V 架构固定为字母“V”编码	0x16
[9:5]	Serial	MRO	系列编码 青稞 V3 系列，固定为数字“3”	0x03
[4:0]	Verision	MRO	版本编码 可以是版本“A”“B”“C”等字母的编码	x

其中厂商编号、版本编号为英文字母，系列编号为数字。字母的编码表如下表所示：
表 8-1 字母映射表

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

其中青稞 V3A 微处理器，该寄存器读返回为零。

硬件实现编号寄存器（mimpid）

该寄存器主要由厂商编码构成，其各位定义如下：

位	名称	访问	描述	复位值
31	Reserved	MR0	保留	1
[30:26]	Vender0	MR0	厂商编码 0 固定为字母“W”编码	0x17
[25:21]	Vender1	MR0	厂商编码 1 固定为字母“C”编码	0x03
[20:16]	Vender2	MR0	厂商编码 2 固定为字母“H”编码	0x08
15	Reserved	MR0	保留	1
[14:8]	Reserved	MR0	保留	0
[7:0]	Minor	MR0	子版本号	0xXX

该寄存器在任何机器实现中均为可读，青稞 V3A 系列处理器中，该寄存器读返回为零。

内核 ID 寄存器 (mhartid)

位	名称	访问	描述	复位值
[31:0]	mhartid	MR0	内核 ID，表明本内核在芯片结构中的内核 ID 号。 注：仅对 V3F 有效	0

机器模式状态寄存器 (mstatus)

该寄存器前文已经描述部分域，其各位定位如下：

位	名称	访问	描述	复位值
31	SD	MR0	用于标记浮点指令是否处于脏状态。 注：仅对 V3F 有效	0
[30:13]	Reserved	MR0	保留	0
[14:13]	FS	MRW	浮点指令集状态，值不为 0 时允许单精度浮点指令 (F) 执行： 00: OFF; 01: Initial; 10: Clean; 11: Dirty。 注：仅对 V3F 有效	0
[12:11]	MPP	MRW	进中断前特权模式	0
[10:9]	VS	MRW	向量指令集状态，值不为 0 时允许向量指令 (V) 执行： 00: OFF; 01: Initial; 10: Clean; 11: Dirty。	0
8	Reserved	MR0	保留	0
7	MPiE	MRW	进中断之前中断使能状态	0
[6:4]	Reserved	MR0	保留	0
3	MiE	MRW	机器模式中断使能	0
[2:0]	Reserved	MR0	保留	0

MPP 域用于保存进入异常或中断前的特权模式，用于退出异常或中断后的特权模式恢复，MIE 为全局中断使能位，当进入异常或中断时，MPIE 的值被更新为 MIE 值，需要注意的是青稞 V3 系列微处理器中，在最后一级嵌套中断前 MIE 不会被更新为 0，以保证机器模式下的中断嵌套继续执行。当退出异常或中断后，微处理器恢复为 MPP 保存的机器模式，并且 MIE 恢复为 MPIE 的值。

青稞 V3 微处理器支持机器模式和用户模式，若需要使微处理器仅工作在机器模式，可在启动文件的初始化中把 MPP 设置为 0x3，即返回后，始终保持在机器模式。

硬件指令集寄存器 (misa)

此寄存器用于指示微处理器的架构和支持的指令集扩展情况，其各位描述如下：

位	名称	访问	描述	复位值
[31:30]	MXL	MRO	机器字长 01: 32 10: 64 11: 128	0x1
[29:26]	Reserved	MRO	保留	0
[25:0]	Extensions	MRO	指令集扩展情况	x

其 MXL 用于指示微处理器的字长，青稞 V3 均为 32 位微处理器，该域固定为 1。Extensions 用于指示微处理器支持扩展指令集详情，每位表示一类扩展，其详细描述如下表所示：

表 8-2 指令集扩展详情

位	名称	描述
0	A	Atomic extension
1	B	Tentatively reserved for Bit-Manipulation extension
2	C	Compressed extension
3	D	Double-precision floating-point extension
4	E	RV32E base ISA
5	F	Single-precision floating-point extension
6	G	Additional standard extensions present
7	H	Hypervisor extension
8	I	RV32I/64I/128I base ISA
9	J	Tentatively reserved for Dynamically Translated Languages extension
10	K	Reserved
11	L	Tentatively reserved for Decimal Floating-Point extension
12	M	Integer Multiply/Divide extension
13	N	User-level interrupts supported
14	O	Reserved
15	P	Tentatively reserved for Packed-SIMD extension
16	Q	Quad-precision floating-point extension
17	R	Reserved
18	S	Supervisor mode implemented
19	T	Tentatively reserved for Transactional Memory extension
20	U	User mode implemented
21	V	Tentatively reserved for Vector extension
22	W	Reserved

23	X	Non-standard extensions present
24	Y	Reserved
25	Z	Reserved

例如青稞 V3A 微处理器，该寄存器值为 0x40101105，即支持的指令集架构为 RV32IMAC，且具有用户模式实现。

机器模式异常基址寄存器（mtvec）

该寄存器用于存储异常或中断处理程序的基地址，并且低两位用于配置向量表的模式和识别方式，详见 3.2 节介绍。

机器模式暂存寄存器（mscratch）

位	名称	访问	描述	复位值
[31:0]	mscratch	MRW	数据暂存	0

该寄存器是一个机器模式下 32 位的可读可写的寄存器，用于临时保存数据。例如，在进入异常或中断处理程序时，将用户堆栈指针 SP 存入该寄存器，并将中断栈指针赋值给 SP 寄存器。退出异常或异常后，从 mscratch 中恢复用户堆栈指针 SP 的值。即可实现中断栈和用户栈的隔离。

机器模式异常程序指针寄存器（mepc）

位	名称	访问	描述	复位值
[31:0]	mepc	MRW	异常程序指针	0

该寄存器用于保存进入异常或中断时的程序指针，其用于在产生异常或中断时保存进入异常前的指令 PC 指针，当处理完异常或中断后，mepc 被作为返回地址，用于异常或中断返回。但是需要注意的是：

- 当发生异常时，mepc 被更新为当前产生异常的指令的 PC 值；
- 当发生中断时，mepc 被更新为下一条指令的 PC 值。

固在处理完异常，需要返回时，应注意修改 mepc 的值，更多的细节内容可参第 2 章异常章节。

机器模式异常原因寄存器（mcause）

位	名称	访问	描述	复位值
31	Interrupt	MRW	中断指示域 0：异常 1：中断	0
[30:0]	Exception Code	MRW	异常编码，详见表 2-1	0

该寄存器主要用于保存产生异常的原因或中断的中断编号，其最高位为 Interrupt 域，用于指示当前发生的是异常还是中断。低位为异常编码，用于指示具体的原因。其详细内容可以参考第 2 章异常章节。

机器模式异常值寄存器（mtval）

位	名称	访问	描述	复位值
[31:0]	mtval	MRW	异常值	0

该寄存器用于保存发生异常时，引起异常的值。其保存的值和时间等详细的内容可以参考第 2 章异常章节。

PMP 配置寄存器（pmpcfg<i>）

该寄存器主要用于物理内存保护单元的配置，该寄存器每 8 位用于配置一个区域的保护，详细定义参考第 4 章。

PMP 地址寄存器 (pmpaddr<i>i>)

该寄存器主要用于物理内存保护单元的地址配置，其是对一个 34 位物理地址的高 32 位的编码，具体的配置方法参考第 4 章。

调试触发选择寄存器 (tselect)

仅对支持硬件断点的微处理器有效，最多支持 4 通道断点，其低 2 位有效。配置各通道断点时，需先通过该寄存器选择对应通道后配置。

位	名称	访问	描述	复位值
[31:2]	Reserved	MRO	保留。	0
[1:0]	TSELECT	MRW	断点通道选择寄存器，配置后即选中对应通道后，可操作 tdata1 和 tdata2 寄存器配置断点信息	X

调试触发数据寄存器 2 (tdata2)

仅对支持硬件断点的微处理器有效，用于保存触发的匹配值。

位	名称	访问	描述	复位值
[31:0]	TDATA2	MRW	用于保存匹配值。	X

触发器信息寄存器 (tinfo)

位	名称	访问	描述	复位值
[31:0]	tinfo	MRO	触发器信息寄存器，对应 TDATA1 寄存器 type 值，type 值为 2，tinfo 的第 2 位置 1。	0x4

注：此寄存器仅对 V3F 有效

调试控制和状态寄存器 (dcsr)

该寄存器用于控制和记录调试模式的运行状态，详细描述参考 7.1 节。

调试模式程序指针 (dpc)

该寄存器用于保存微处理器进入调试模式之后将要执行的下一条指令的地址，其值根据进入调试的原因不同，更新的规则也不相同。详细描述参考 7.1 节。

调试触发数据寄存器 1 (tdata1)

仅对支持硬件断点的微处理器有效，微处理器仅支持指令地址和数据地址断点，其中 tdata1 寄存器的位 TYPE 为固定值 2，其它位符合调试标准中 mcontrol 的定义。

位	名称	访问	描述	复位值
[31:28]	TYPE	MRO	断点类型定义，mcontrol 类型。	0x2
27	DMODE	MRO	0：机器模式和调试模式均可修改触发器的相关寄存器； 1：仅调试模式可修改触发器的相关寄存器。	1
[26:21]	MASKMAX	MRO	当 MATCH = 1 时，允许匹配的最大指数幂范围，即最大允许匹配 2^{31} 字节范围。	0x1F
[20:13]	Reserved	MRO	保留。	0
12	ACTION	MRW	设置触发断点时采取的处理模式： 0：触发时进入到断点回调中断； 1：触发时进入到调试模式。	0
[11:8]	Reserved	MRO	保留。	0

7	MATCH	MRW	匹配策略配置： 0：触发值和 TDATA2 相等时匹配； 1：触发值和 TDATA2 的高 M 位相等时匹配，其中 $M = 31 - N$ ，N 为 TDATA2 的第一个 0 的索引（从低位开始）。	0
6	M	MRW	M 模式下触发器使能： 0：M 模式下关闭触发器； 1：M 模式下使能触发器。	0
[5:4]	Reserved	MRO	保留。	0
3	U	MRW	U 模式下触发器使能： 0：U 模式下关闭触发器； 1：U 模式下使能触发器。	0
2	EXECUTE	MRW	指令读地址触发使能： 0：关闭； 1：使能。	0
1	STORE	MRW	数据写地址触发使能： 0：关闭； 1：使能。	0
0	LOAD	MRW	数据读地址触发使能： 0：关闭； 1：使能。	0

调试模式暂存寄存器（dscratch0-1）

该组寄存器用于调试模式下临时存储数据。

位	名称	访问	描述	复位值
[31:0]	dscratch	DRW	调试模式数据暂存值	0

调试寄存器 0（dbgmcu_0）

位	名称	访问	描述	复位值
[31:8]	dbg_mode_stop	MRW	进入调试模式时注册模块停止工作： 1：注册的外设在内核进入调试模式时停止工作； 0：注册的外设在内核进入调试模式时正常工作。	0
[7:0]	dbg_link_stop	MRW	进入连接调试器时注册模块停止工作： 1：注册的外设在连接调试器时停止工作； 0：注册的外设在连接调试器时正常工作。	0

浮点累计异常标记寄存器（fflags）

位	名称	访问	描述	复位值
[31:5]	Reserved	RO	保留。	0

[4:0]	FFlags	RW	累计异常标记，用于表示自上次重置以来（复位或软件清除）在浮点运算指令上出现的异常。			0
			name	bit	description	
			NV	4	非法操作	
			DZ	3	除数为零	
			OF	2	向上溢出	
			UF	1	向下溢出	
			NX	0	不精确	

注：此寄存器仅对 V3F 有效

浮点舍入模式寄存器（frm）

位	名称	访问	描述	复位值
[31:3]	Reserved	RO	保留。	0
[2:0]	Frm	RW	浮点操作舍入模式寄存器： 000：向最近值舍入（处于中间值时向偶数方向舍入）； 001：向零舍入； 010：向下舍入； 011：向上舍入； 100：向最近值舍入（处于中间值时向最大量级舍入）； 101、110：保留； 111：动态舍入，根据指令中的 FRM 位配置值选择舍入模式。	0

注：此寄存器仅对 V3F 有效

浮点控制和状态寄存器（fcsr）

位	名称	访问	描述	复位值
[31:8]	Reserved	RO	保留。	0
[7:5]	Frm	RW	浮点操作舍入模式寄存器。	0
[4:0]	FFlags	RW	累计异常标记。	0

注：此寄存器仅对 V3F 有效

定点饱和状态寄存器（vxsat）

位	名称	访问	描述	复位值
[31:1]	Reserved	URW	保留。	0
0	VXSAT	URW	定点饱和状态寄存器，当执行的定点运算结果必须输出为饱和值时置 1。	0

注：此寄存器仅对 V3V 有效

定点舍入寄存器（vxrm）

位	名称	访问	描述	复位值
[31:2]	Reserved	URW	保留。	0

[1:0]	VXRM	URW	定点操作舍入寄存器，用于控制定点操作的舍入模式： 00：向最近值舍入（中间值向上舍入）； 01：向最近值舍入（中间值向偶数舍入）； 10：向下舍入； 11：向奇数舍入。	0
-------	------	-----	--	---

注：此寄存器仅对 V3V 有效

向量控制和状态寄存器（vcsr）

位	名称	访问	描述	复位值
[31:3]	Reserved	URW	保留。	0
[2:1]	VXRM	URW	定点操作舍入寄存器，用于控制定点操作的舍入模式： 00：向最近值舍入（中间值向上舍入）； 01：向最近值舍入（中间值向偶数舍入）； 10：向下舍入； 11：向奇数舍入。	0
0	VXSAT	URW	定点饱和状态寄存器，当执行的定点运算结果必须输出为饱和值时置 1。	0

注：此寄存器仅对 V3V 有效

计数器屏蔽寄存器（mcount_inhibit）

位	名称	访问	描述	复位值
[31:3]	Reserved	MRO	保留。	0
2	IR	MRW	指令计数器屏蔽寄存器： 1：屏蔽指令计数器，minstret 和 unstret 停止计数； 0：不屏蔽指令计数器，minstret 和 unstret 正常计数。	0
1	Reserved	MRO	保留。	0
0	CY	MRW	周期计数器屏蔽寄存器： 1：屏蔽指令计数器，mcycle 和 ucycle 停止计数； 0：不屏蔽周期计数器，mcycle 和 ucycle 正常计数。	0

注：此寄存器仅对 V3V 有效

机器周期计数器寄存器（mcycle）

位	名称	访问	描述	复位值
[31:0]	MCYLE	MRW	机器周期计数器，用于计数机器工作的时钟周期，mcount_inhibit 寄存器 CY 位为 0 时计数，调试模式中计数器停止时不计数。	0

注：此寄存器仅对 V3V 有效

机器指令计数器寄存器（minstret）

位	名称	访问	描述	复位值
[31:0]	MINSTRET	MRW	指令计数器，用于计数机器正确执行完成的指令数，MCOUNT_INHIBIT 寄存器 IR 位为 0 时计数，调试模式中计数器停止时不计数。	0

注：此寄存器仅对 V3V 有效

用户模式周期计数器寄存器 (ucycle)

位	名称	访问	描述	复位值
[31:0]	UCYLE	RO	用户模式周期计数器，用于计数机器工作的时钟周期，mcount_inhibit 寄存器 CY 位为 0 时计数，调试模式中计数器停止时不计数。	0

注：1. 用户模式周期计数器，功能同 mcycle 寄存器，用于在用户模式下读取机器运行周期数。

2. 此寄存器仅对 V3V 有效。

用户模式指令计数器寄存器 (uinstret)

位	名称	访问	描述	复位值
[31:0]	uinstret	RO	指令计数器，用于计数机器正确执行完成的指令数，MCOUNT_INHIBIT 寄存器 IR 位为 0 时计数，调试模式中计数器停止时不计数。	0

注：1. 用户模式指令计数器，功能同 minstret 寄存器，用于在用户模式下读取机器正确执行的指令数。

2. 此寄存器仅对 V3V 有效。

元素体长度寄存器 (vl)

位	名称	访问	描述	复位值
[31:7]	Reserved	URW	保留。	0
[6:0]	VL	URO	向量体长度寄存器，用于存储运算中的使用的元素长度，寄存器值只能被 vset 指令修改，赋值规则见《RISC-V “V” vector Extension 1.0》手册。	0

注：此寄存器仅对 V3V 有效

向量数据类型寄存器 (vtype)

位	名称	访问	描述	复位值
[31:6]	Reserved	URW	保留。	0
[5:3]	VSEW	URO	向量宽度寄存器，用于选择向量和元素的宽度，仅 VSET 指令可配置： 000: 8bit; 001: 16bit; 010: 32bit; 011: 64bit; 1xx: 保留。 注：配置 eew 为 64bit 需使能 vcontrol 相关配置。	0
[2:0]	VLMUL	URO	向量组分配寄存器，用于选择向量运算	0

			中的分组信息，仅 VSET 指令可配置： 000: 1; 001: 2; 010: 4; 011: 8; 100: 保留; 101: 1/8; 110: 1/4; 111: 1/2。	
--	--	--	---	--

注：此寄存器仅对 V3V 有效

向量寄存器字节长度寄存器 (vlenb)

位	名称	访问	描述	复位值
[31:0]	VLENB	URO	向量寄存器组字节长度寄存器，本内核向量寄存器长度 64bit，字节长度 8，固定值	0x8

注：此寄存器仅对 V3V 有效

自定义向量控制寄存器 (vcontrol)

位	名称	访问	描述	复位值
31	VEC_ILL	URW1C	向量指令异常标记，当向量指令产生异常时置位，当 NMI_EN 寄存器被使能时，向量异常将进入 NMI 异常。	0
[30:24]	Reserved	URO	保留。	0
[23:22]	NEST_INT_PTR	URO	中断嵌套深度： 00: 无中断； 01: 一级中断嵌套； 10: 二级中断嵌套； 11: 三级中断嵌套。	0
[21:20]	VEC_INT_PTR	URW	向量流水线中断嵌套深度： 00: 无中断； 01: 一级中断嵌套； 10: 二级中断嵌套； 11: 三级中断嵌套。 当中断服务函数中未使用向量指令时，向量流水中断嵌套深度保持前值。	0
[19:16]	VEC_REG_STATE	URW	向量寄存器状态寄存器，共 4 位对应最大四层向量寄存器状态深度，值为 1 表示对应中断深度向量寄存器状态为脏状态，即执行过向量指令导致向量寄存器值变化。	0
[15:10]	Reserved	URO	保留。	0
9	VEC_ILL_INFO	URW	向量异常信息保存使能，置 1 后 VCAUSE 和 VTVAL 寄存器将保存向量异常信息。	1
8	Reserved	URO	保留，不可写 0。	1

[7:4]	Reserved	URO	保留。	0
3	LSU_WAX_ON	URW	强制写顺序使能，强制写操作按指令顺序执行。	1
2	LSU_RAW_ON	URW	强制读顺序使能，强制读操作按指令顺序执行。	1
1	VEC_HW_PP_ABI	URW	向量寄存器硬件堆栈保护 ABI 配置寄存器： 1：调用变体 ABI，硬件堆栈保护 V0，V8-V23 寄存器； 0：调用标准 ABI，硬件堆栈保护 V0-V31 寄存器。 注：ABI 调用可参考《RISC-V ABIS SPECIFICATION V1.1》	0
0	VEC_HW_PP_EN	URW	向量寄存器硬件堆栈保护使能： 1：开启向量堆栈保护功能； 0：关闭向量堆栈保护功能。	0

注：此寄存器仅对 V3V 有效

自定义向量寄存器压栈基址寄存器（vppaddr）

位	名称	访问	描述	复位值
[31:0]	VPPADDR	URW	向量寄存器堆栈保护内存地址寄存器，用于配置向量寄存器堆栈保护地址，低 10bit 固定为 0，内存中 1KB 对齐。	0x20000000

注：此寄存器仅对 V3V 有效

自定义向量异常源寄存器（vcause）

位	名称	访问	描述	复位值
[31:0]	VCAUSE	URO	向量指令异常原因寄存器，当 VEC_ILL_INFO 使能时，用于保存向量异常的异常原因。 注：调试模式将导致异常原因信息丢失，无法在调试模式中使用	0

注：此寄存器仅对 V3V 有效

自定义向量异常自陷信息寄存器（vtval）

位	名称	访问	描述	复位值
[31:0]	VTVAL	URO	向量指令自陷信息寄存器，当 VEC_ILL_INFO 使能时，用于保存向量异常的 PC 值。 注：调试模式将导致异常原因信息丢失，无法在调试模式中使用。	0

注：此寄存器仅对 V3V 有效

8.3 用户自定义 CSR 寄存器

用户模式全局中断使能寄存器（gintennr）

该寄存器用于控制全局中断的使能与屏蔽，机器模式下全局中断的使能与屏蔽可以通过寄存器 `mstatus` 中的 `MIE`，`MPIE` 位控制，该寄存器用户模式下无法操作。而全局中断使能寄存器 `gintennr` 是 `mstatus` 中 `MIE` 和 `MPIE` 的映射，用户模式下可以通过操作 `gintennr`，用于 `MIE` 和 `MPIE` 的置位和清零，详见 3.2 节相关描述。

注：全局中断不包括不可屏蔽中断 `NMI` 和异常

中断系统控制寄存器（`intsyscr`）

该寄存器主要用于配置中断嵌套深度、硬件压栈等相关功能，详见 3.2 节相关描述。

微处理器配置寄存器（`corecfr`）

该寄存器用于控制中断溢出后是否允许进入 `NMI` 中断，以及执行 `fence` 指令时是否清除中断请求，具体定义参考 3.2 节相关描述。

中断嵌套控制寄存器（`inestcr`）

该寄存器用于指示中断嵌套状态及是否溢出，以及控制最大嵌套级数，具体定义参考 3.2 节相关描述。

第 9 章 V3 系列内核型号与子版本说明

V3 处理器当前主要使用的是 V3B、V3C、V3F 版本，其中 V3C 版本使用最多。

	V3A	V3B	V3C	V3F
AMO	有	无	无	有
MRS（移位乘法自定义指令）	无	有	无	无
中断优先级寄存器宽度	4	2	3	3
PMP	有	无	有	有
LSU_TRIGGER	无	无	有	有

MIMPID 是一个 csr 寄存器，用于记录子版本信息：

mimpid[7:0]=1；仅 CH32M030，CH585。

mimpid[7:0]=2；支持软件复位（即中断模块 FPIC_CFGR 寄存器）

软件中断（增加 STK_CTLR[31]为软件中断标志位，功能同 STK_SR[31]）；

支持中断嵌套使能和屏蔽；

支持抢占优先级位宽配置；

mimpid[7:0]=3；新增自定义延时指令；

支持 zicond 指令。

第 10 章 附录 1 延时指令

自定义延时指令是以单指令开销为代价的精确延时指令，可延时指定匹配的指令精确至时钟周期精度，避免定时器，延时等操作中指令跳转执行的不确定性，指令可自动匹配指令编码、配置延时周期、时钟分频系数，使用灵活，在精确控制、流程管理、低开销延时等应用中有明显优势。

延时指令编码格式

位	名称	描述
[31:20]	imm	延时立即数，根据配置可表示延时的时钟周期数或分频后的周期数。
[19:15]	rs1	匹配的 rs1 寄存器编码。
[14:12]	func3	001b。
[11: 9]	match	匹配的指令类型，在延时未结束时匹配的指令类型无法执行 000: 保留； 001: 匹配 load 指令； 010: 匹配 store 指令； 011: 匹配 load/store 指令； 100: 匹配 delay 指令； 101: 保留； 110: 匹配所有指令（流水线暂停）； 111: 匹配指定 func3 和 opcode 的指令（匹配值由 csr 寄存器配置）。
8	div	使用主频时钟或分频时钟计数 1: 使用分频周期计数（分频系数由 csr 寄存器配置）； 0: 使用主频周期计数。
7	sel	延时数选择： 1: rs1+rs2； 0: rs1 + imm。
[6:0]	func7	0001011b。

注：以 sp 寄存器为基地址的读写指令执行不受匹配 load/store 指令（match=001/010/011）限制。

延时指令控制寄存器（U_NONS_DLY_0）

CSR 地址：0x8C0

位	名称	访问	描述	复位值
[31:24]	dly_mask	URW	dly 指令 match=111 时匹配的指令参数掩膜。	x
[23:16]	dly_data	URW	dly 指令 match=111 时匹配的 {func, opcode}。	x
15	CPU_dly_busy_sta	UR0	内核处于延时状态标志。	0
14	dly_tout	RW1Z	超时信息，在延时范围内无匹配的指令请求。	0
13	dly_ie	URW	延时中断请求，使能后如果产生超时情况，即产生一个软件中断请求。	0
12	dly_priv	URW	1: 机器模式和用户模式均允许匹配	0

			0: 在用户模式下执行的 delay 操作不匹配机器模式中的指令。	
[11:10]	Reserved	UR0	保留。	0
[9:0]	dly_freq	URW	延时计数分频系数配置，若配置为主频时钟频率，可实现 us 为单位延时。	x